

EE 446: TinyML

Lab 2: TinyML Hardware and Sensor Setup

Spring 2026

Instructor: Dinuka Sahabandu (sdinuka@uw.edu)

Assistant Teaching Professor

Department of Electrical and Computer Engineering

University of Washington



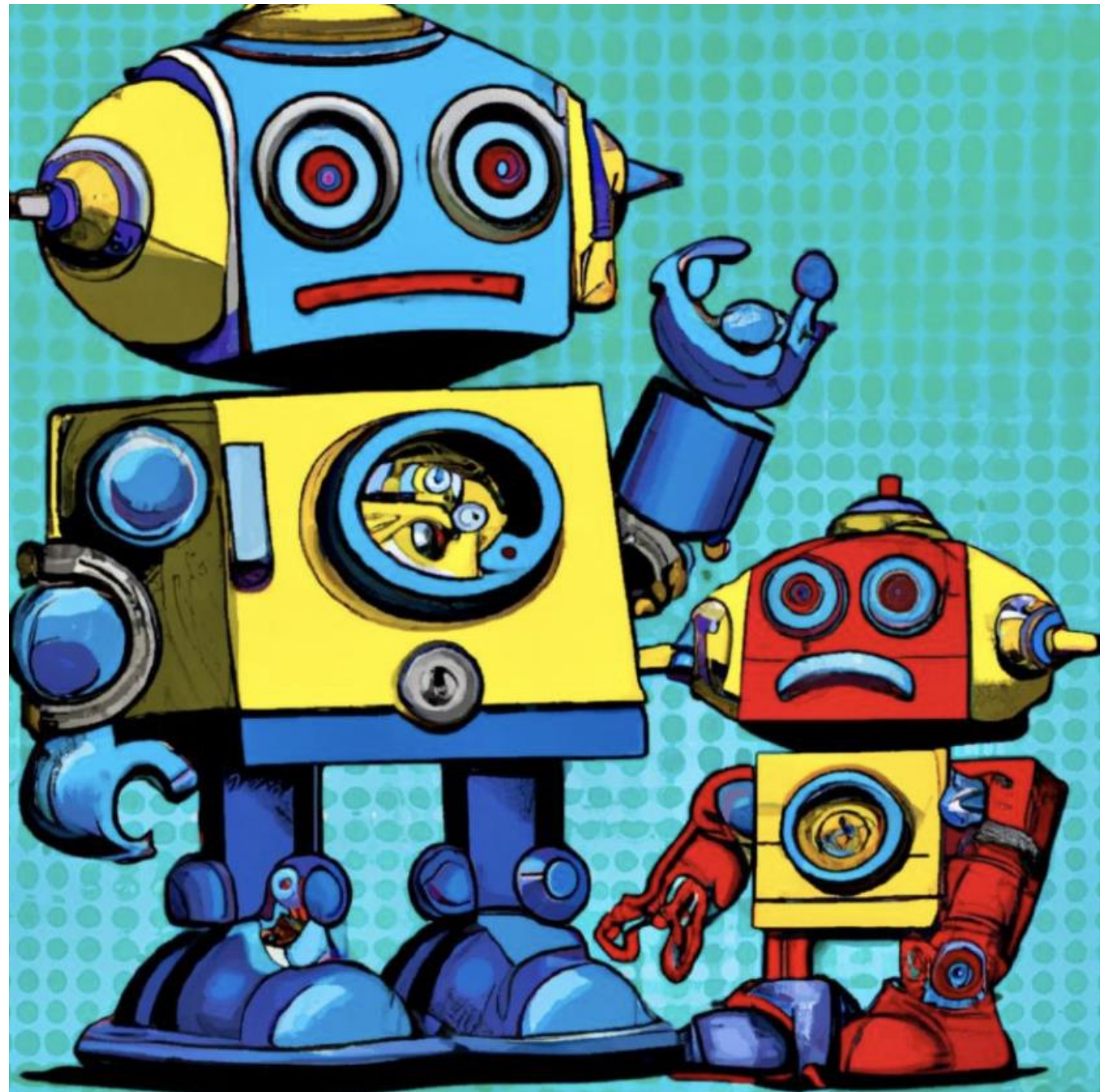
ELECTRICAL & COMPUTER
ENGINEERING

UNIVERSITY of WASHINGTON

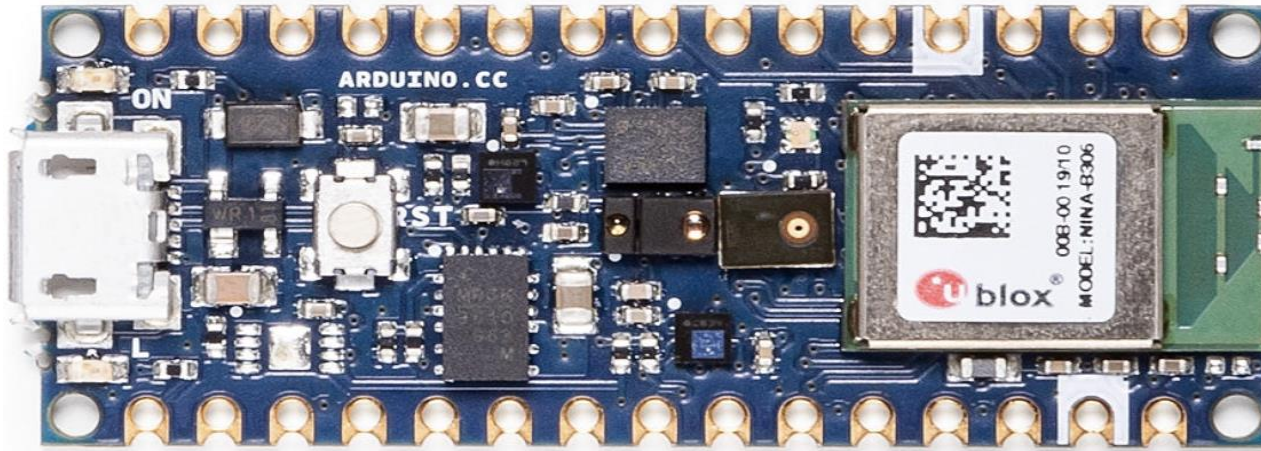


Lab #2

- Setting up TinyML hardware
- Setting up Arduino Desktop IDE
- The Arduino Blink Example
- Testing the Sensors



Components of TinyML Kit



ARDUINO NANO 33 BLE SENSE Rev2 (*provided*)



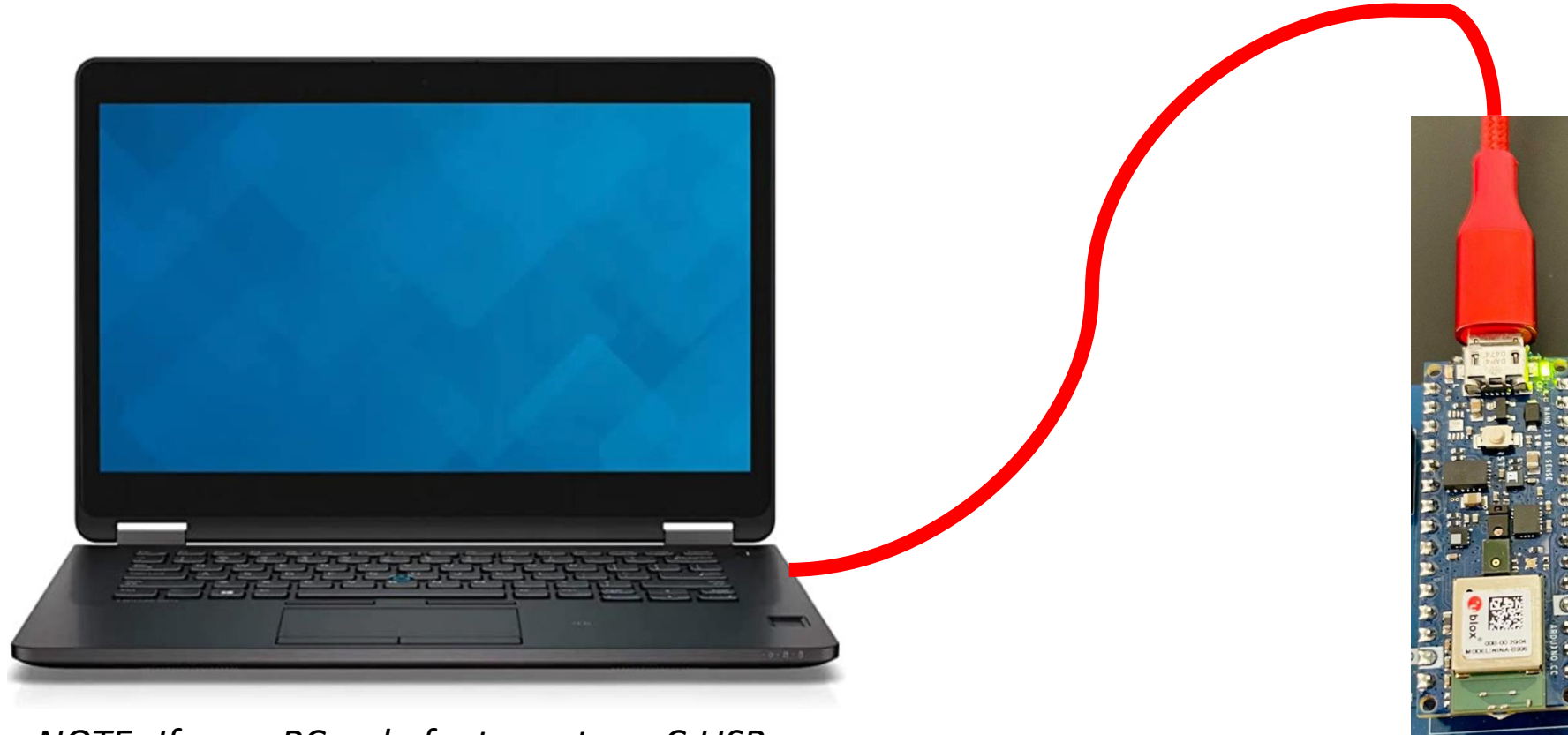
USB A – Micro USB Cable (1m) – (*Not provided; purchase required*)



1 OV7675 Camera
(*Not provided; optional purchase*)

Setting up TinyML Kit

3. Use the USB cable (type-A to microB) to connect the Nano 33 BLE Sense to your machine



NOTE: If your PC only features type-C USB ports, you will need to obtain an adaptor

Task 1 — Hardware Bring-Up and Basic Connection

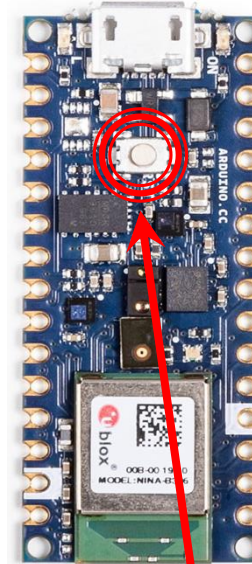
Goal: Confirm that the Arduino Nano 33 BLE Sense Rev2 powers on correctly, is recognized by the computer, and is ready for Arduino IDE setup.

What you need

- Arduino Nano 33 BLE Sense Rev2
- Micro-USB cable capable of both power and data transfer
- Computer with an available USB port

Step-by-step instructions

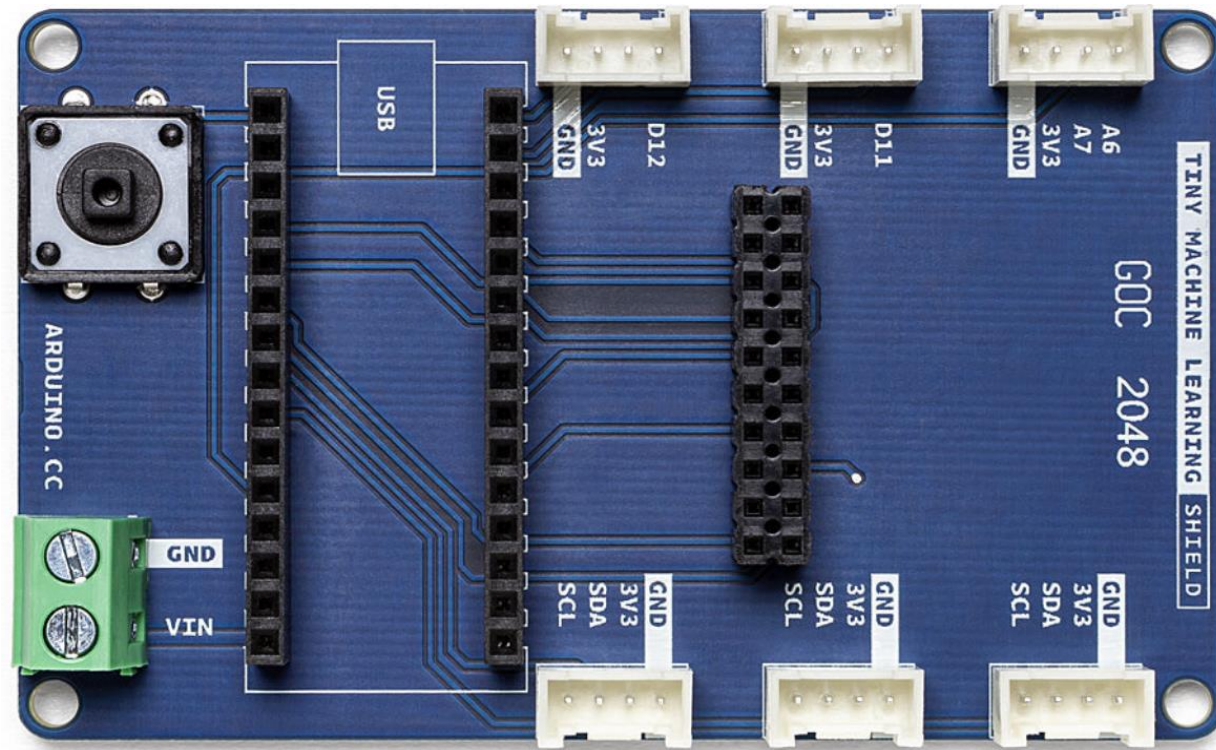
1. Inspect the board and make sure there is no visible damage.
2. Use the micro-USB cable to connect the board directly to your computer.
3. Check whether the power LED turns on.
4. Wait a few seconds for the computer to detect the board.
5. If the board is not detected later in Arduino IDE, double-tap the reset button to place the board into bootloader mode.



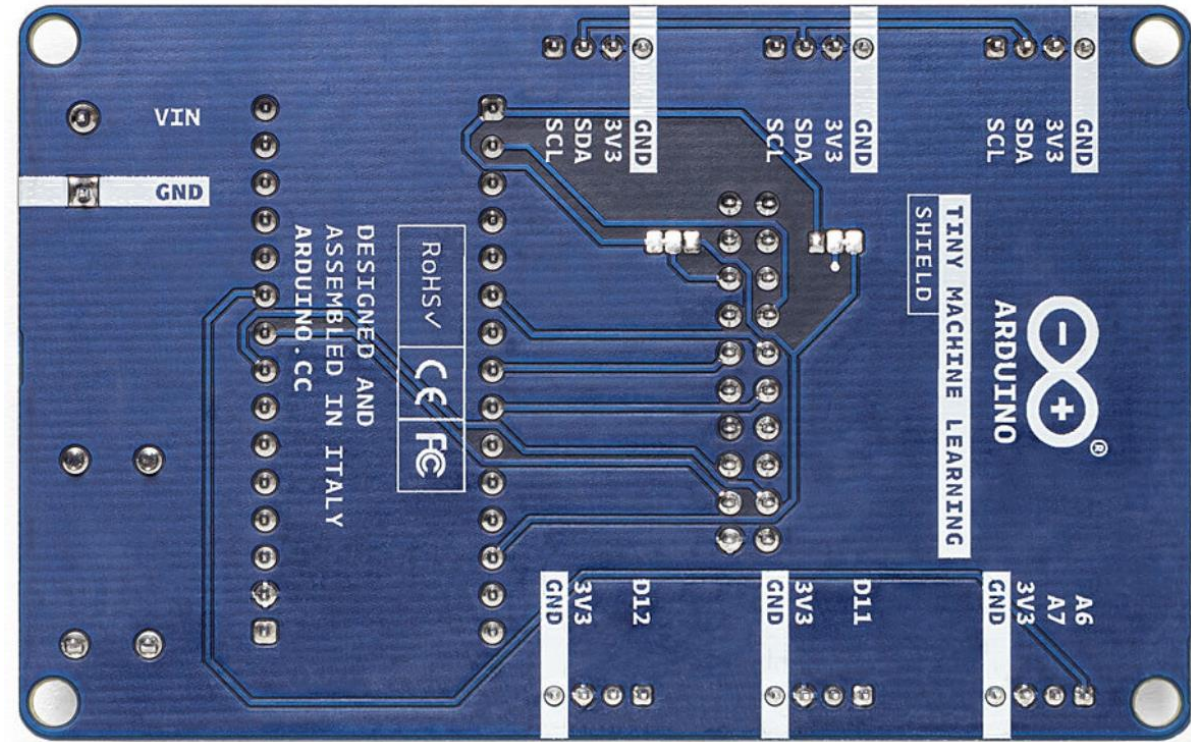
**Reset button on Nano BLE
Sense board**
Double tap to RESET

[Optional] Components of TinyML Kit

(**Not provided**; optional purchase required to connect the camera)



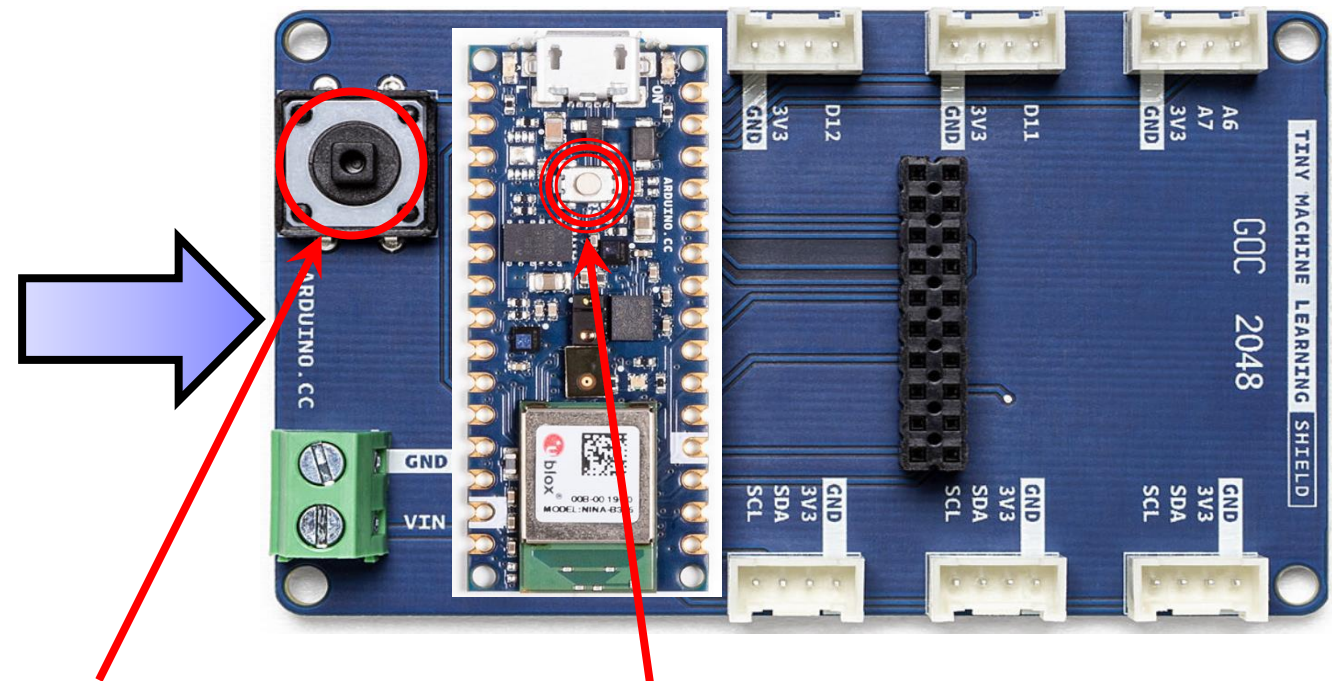
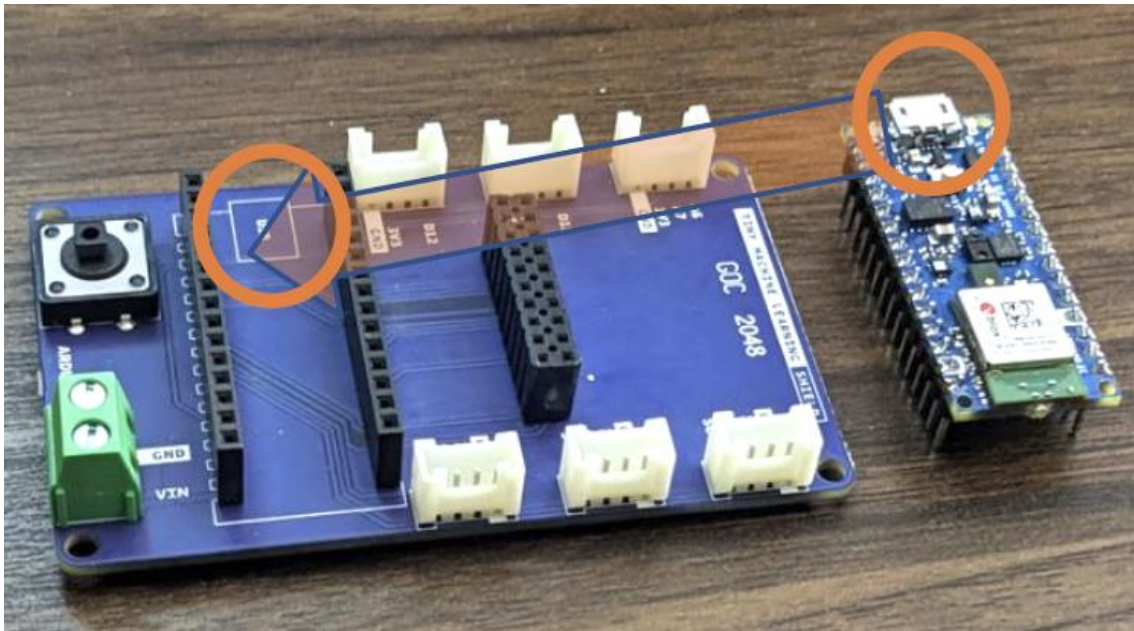
Tiny Machine Learning Shield (Front)



Tiny Machine Learning Shield (Back)

[Optional] Setting up TinyML Kit with Camera

1. Slot the Nano 33 BLE Sense board into the Tiny Machine Learning shield

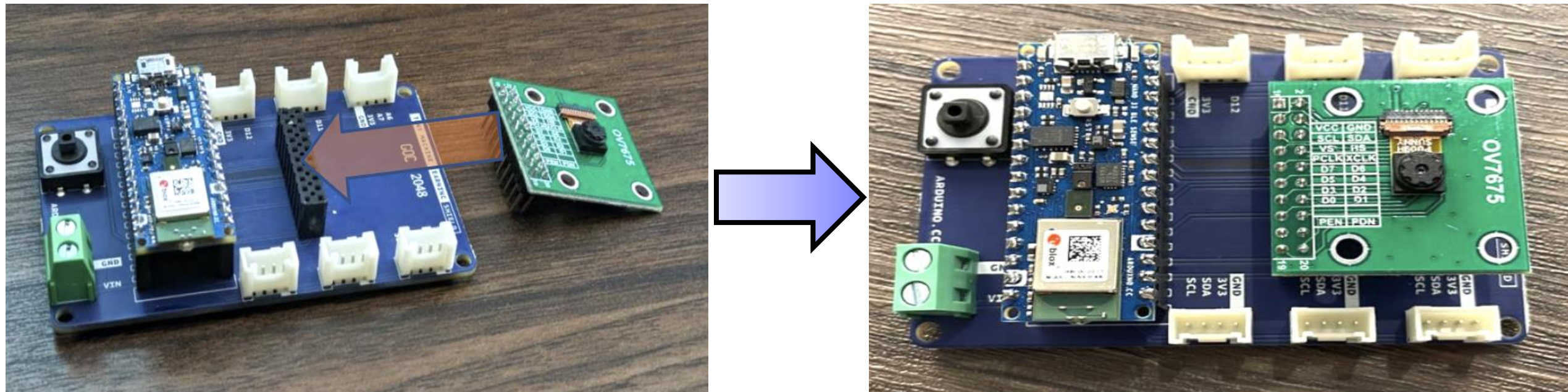


**Control button
on the shield**

**Reset button on Nano BLE Sense board;
Double tap to RESET**

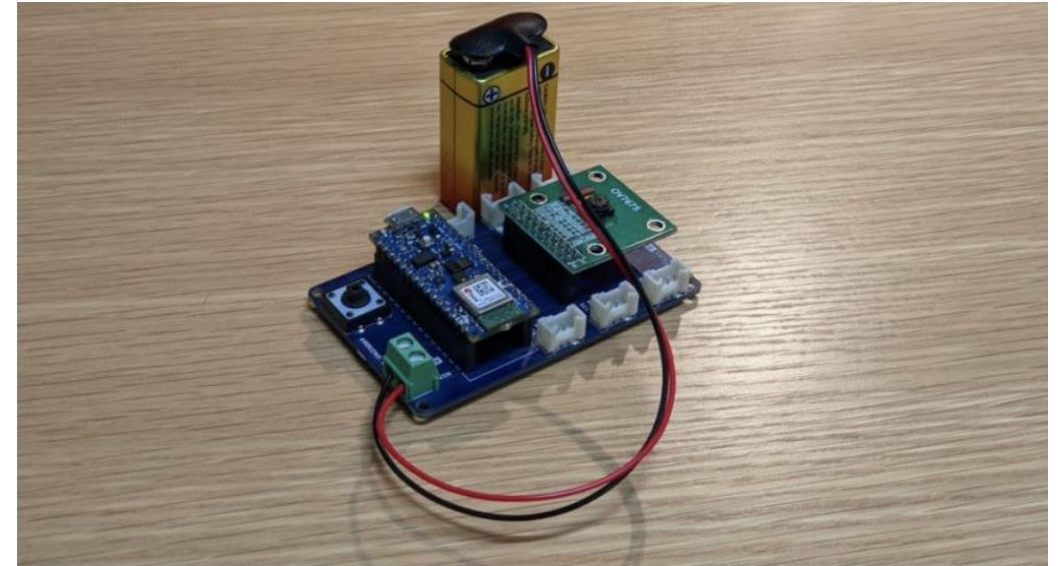
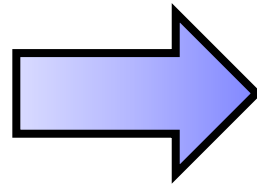
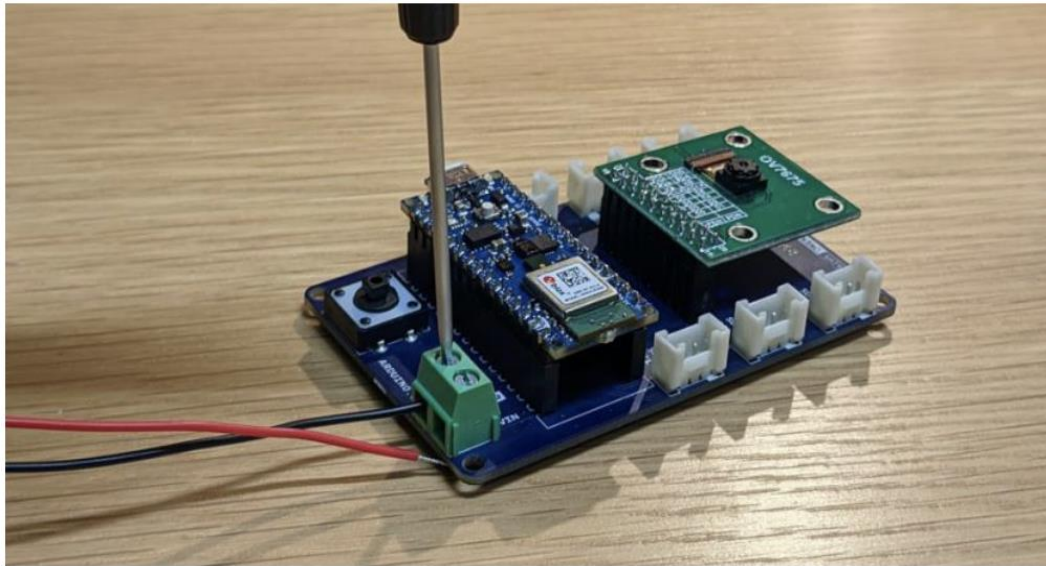
[Optional] Setting up TinyML Kit with Camera

2. Slot the OV7675 camera module into the shield Tiny Machine Learning shield



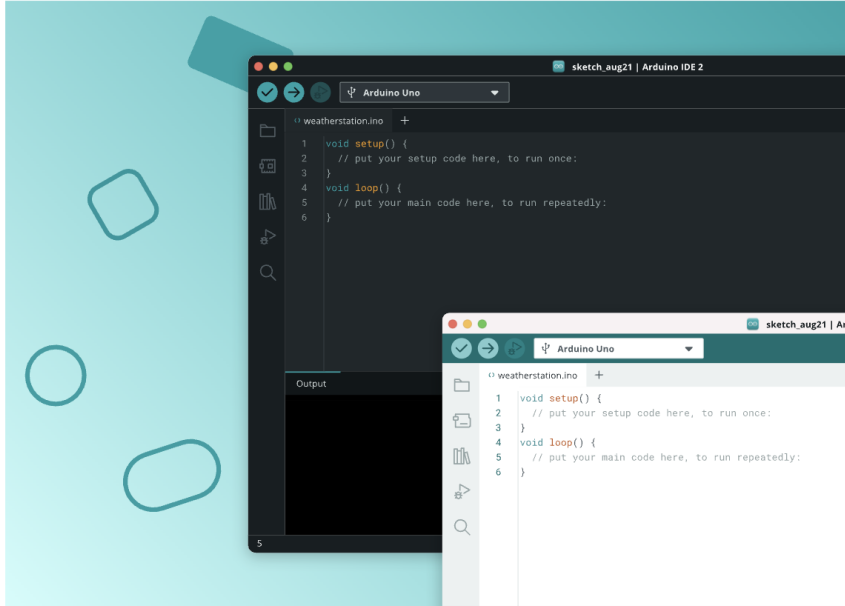
[Optional] Setting up TinyML Kit with Battery

- Using a 9V battery to power Nano 33 BLE Sense
 - Screw down a wire leading from the negative battery terminal (black) to GND. (Most < 3mm flat-head screwdrivers will suffice here).
 - Repeat this process for the positive battery terminal (red) to VIN. And that's it you're all set to power your Arduino from a battery!



Setting up Arduino Desktop IDE

- **Integrated Development Environment (IDE):** An application that facilitates software development
- **Arduino Desktop IDE:** facilitate software development for microcontroller boards, in **C++**



**Arduino IDE 2.3.8**
Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

macOS Apple Silicon 11 Big Sur or newer (64-bit) **DOWNLOAD**

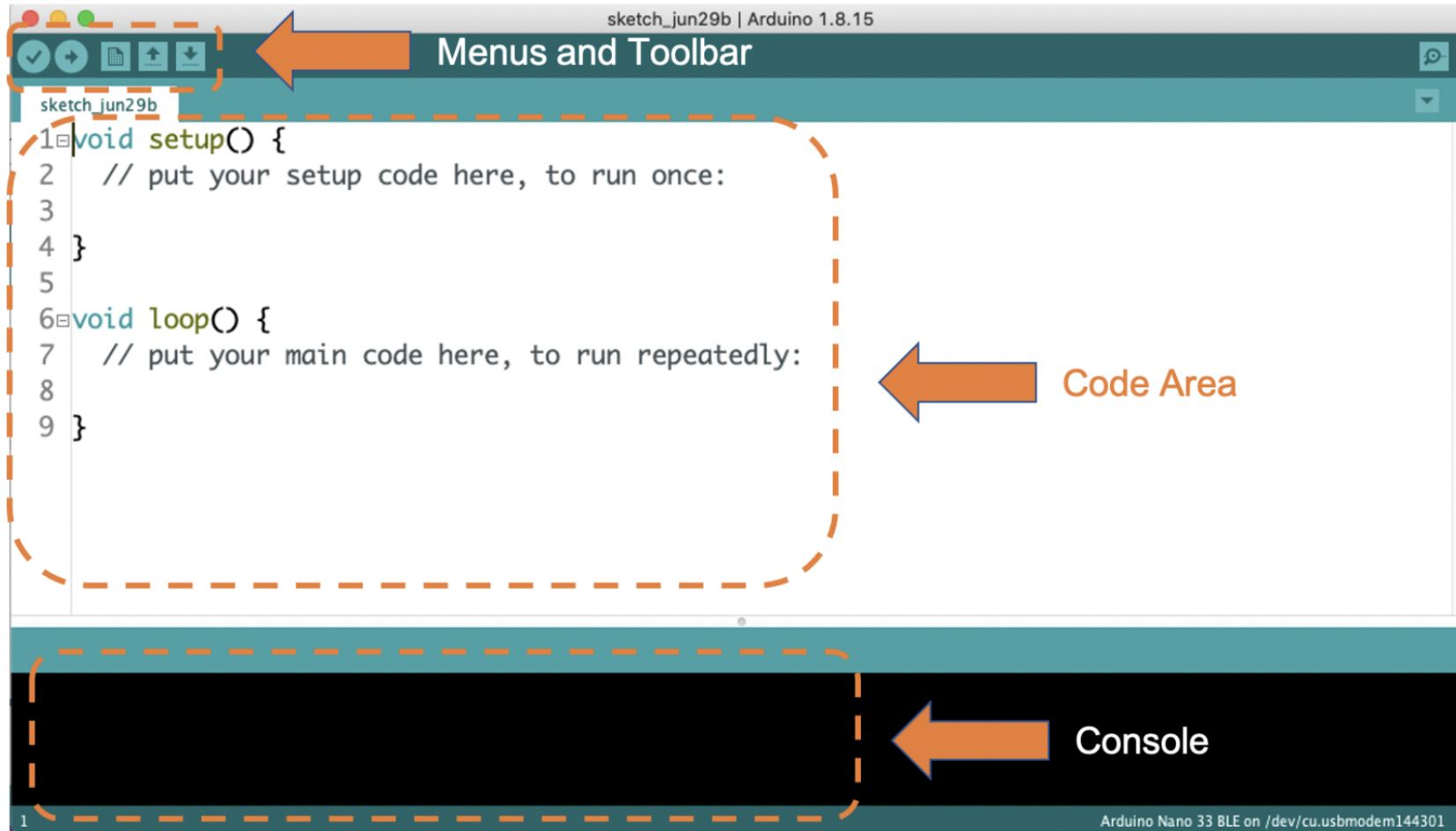
 **Nightly Builds**
Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

- **Download link:** <https://www.arduino.cc/en/software>

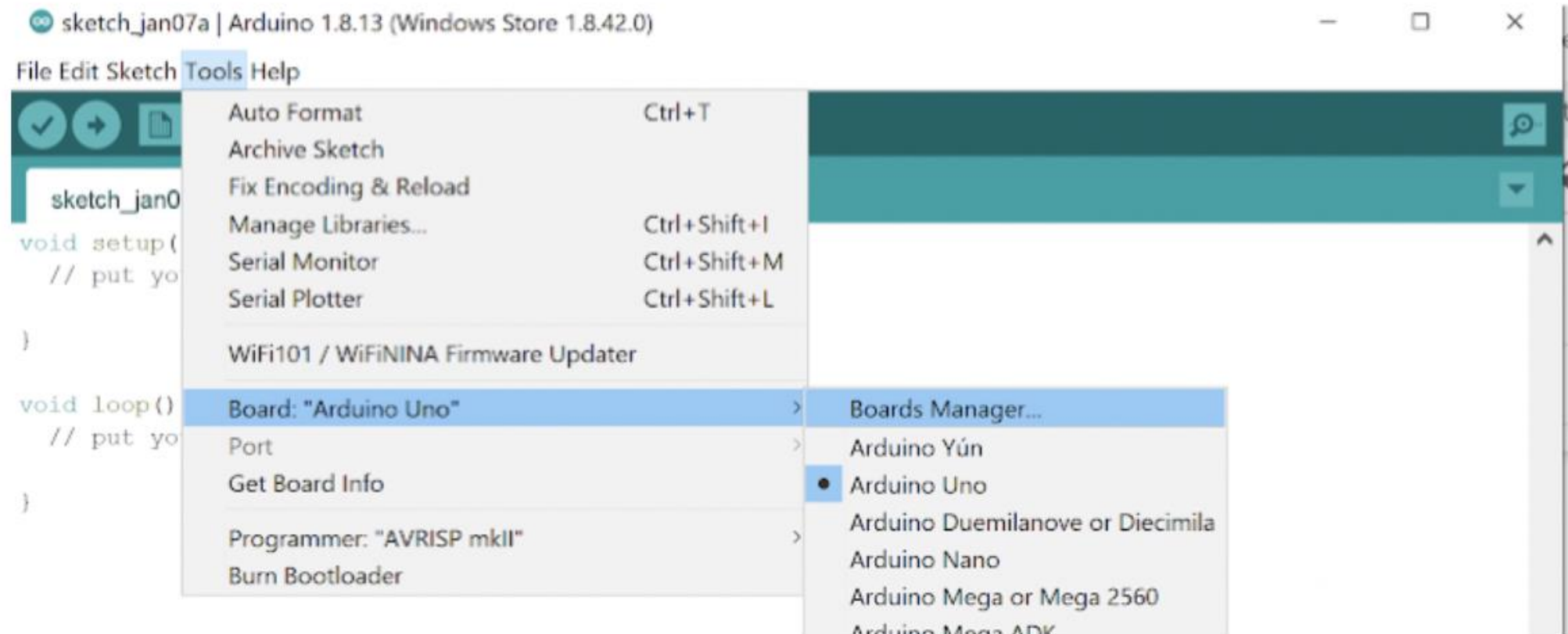
Setting up Arduino Desktop IDE

A Quick Tour of the IDE

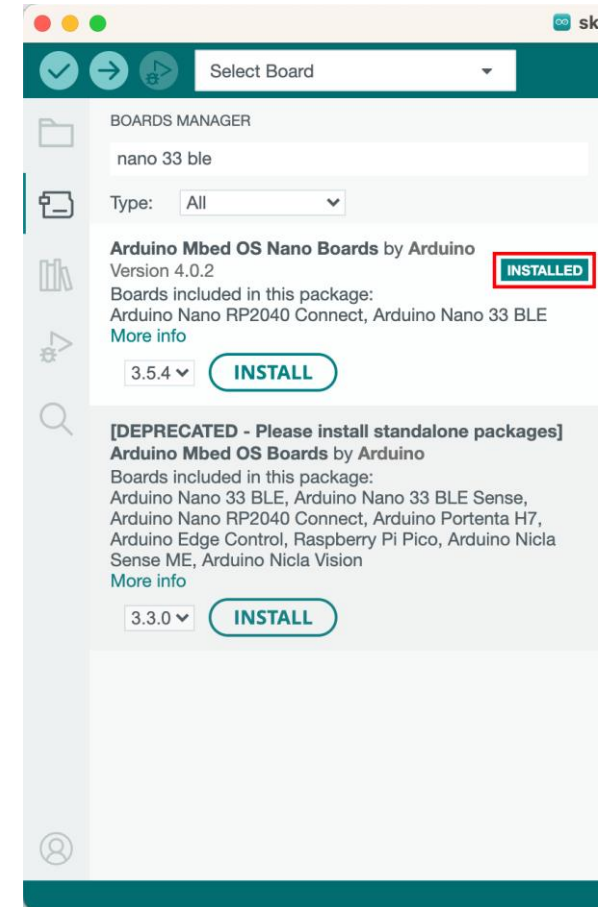
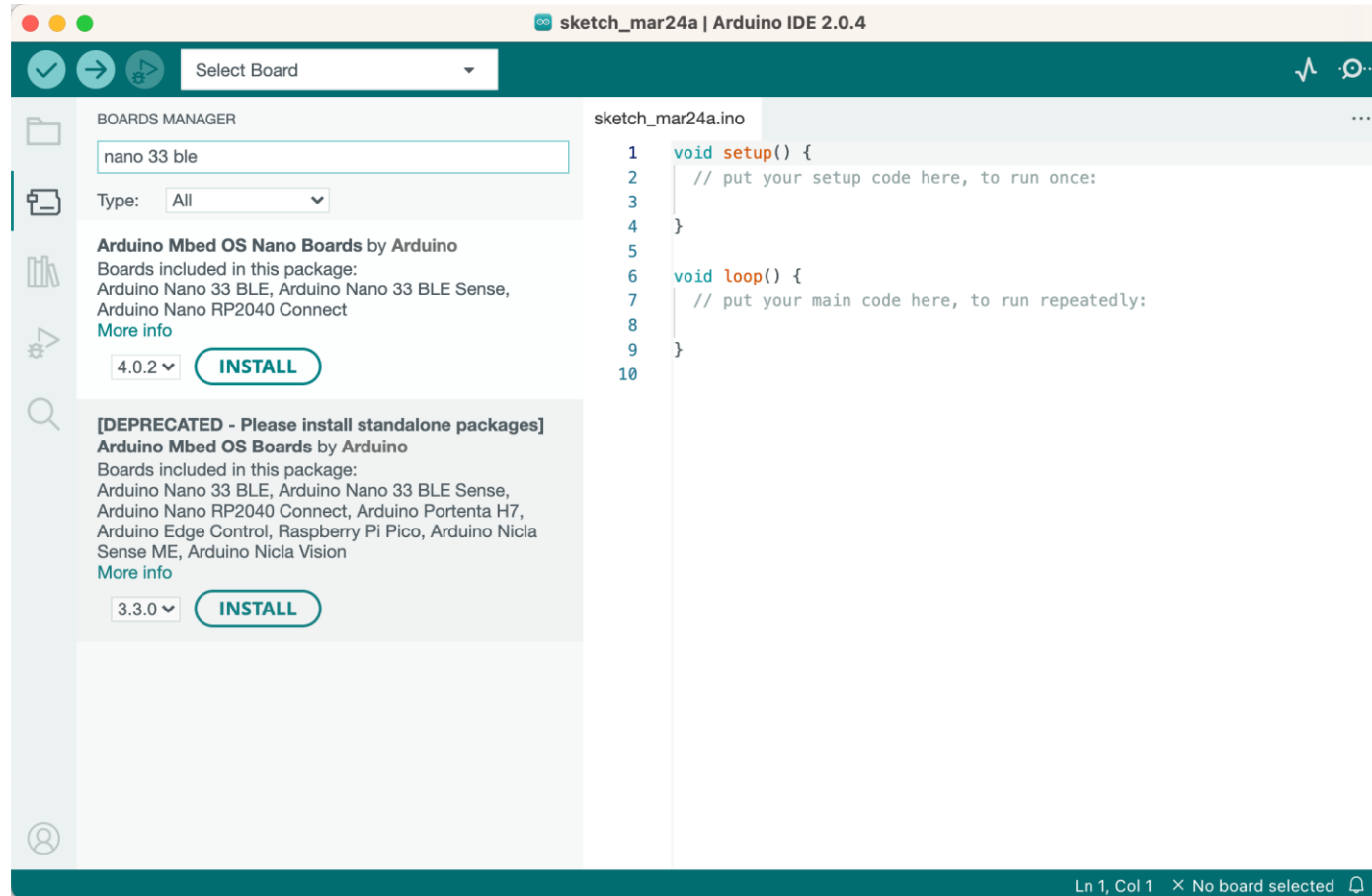


Setting up Arduino Desktop IDE

- Installing the Board Files for the Nano 33 BLE Sense

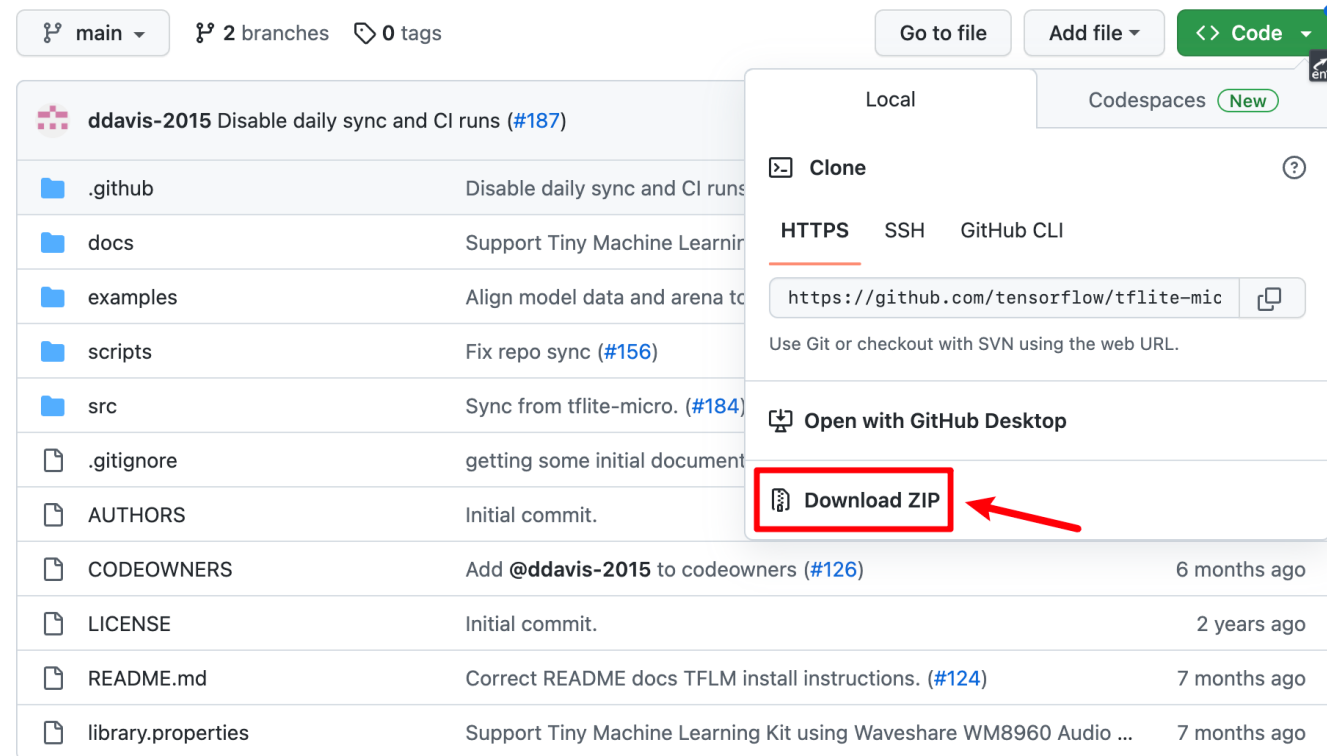


Setting up Arduino Desktop IDE



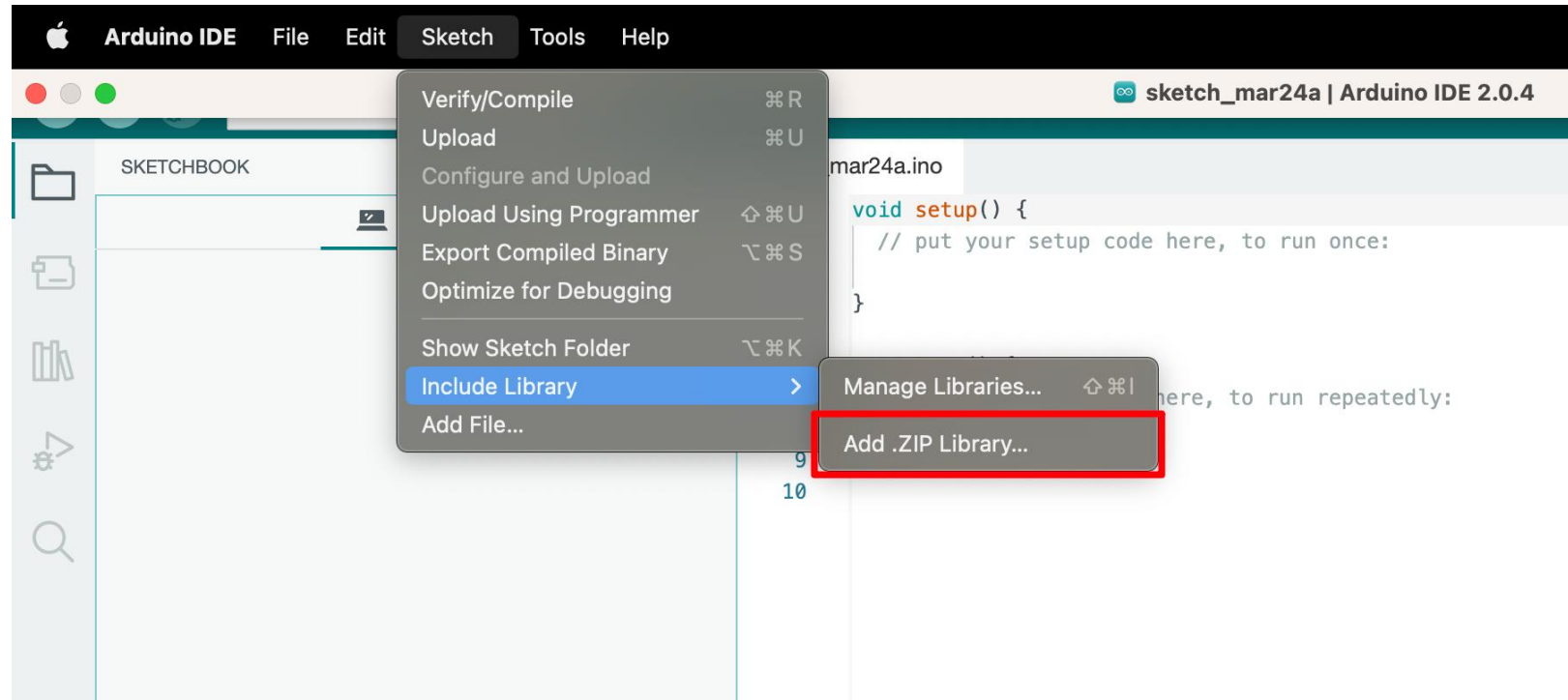
Setting up Arduino Desktop IDE

- Installing the Libraries needed
- **1. TensorFlow Lite Micro Library for Arduino**
- <https://github.com/tensorflow/tflite-micro-arduino-examples#how-to-install>



Setting up Arduino Desktop IDE

1. TensorFlow Lite Micro Library for Arduino

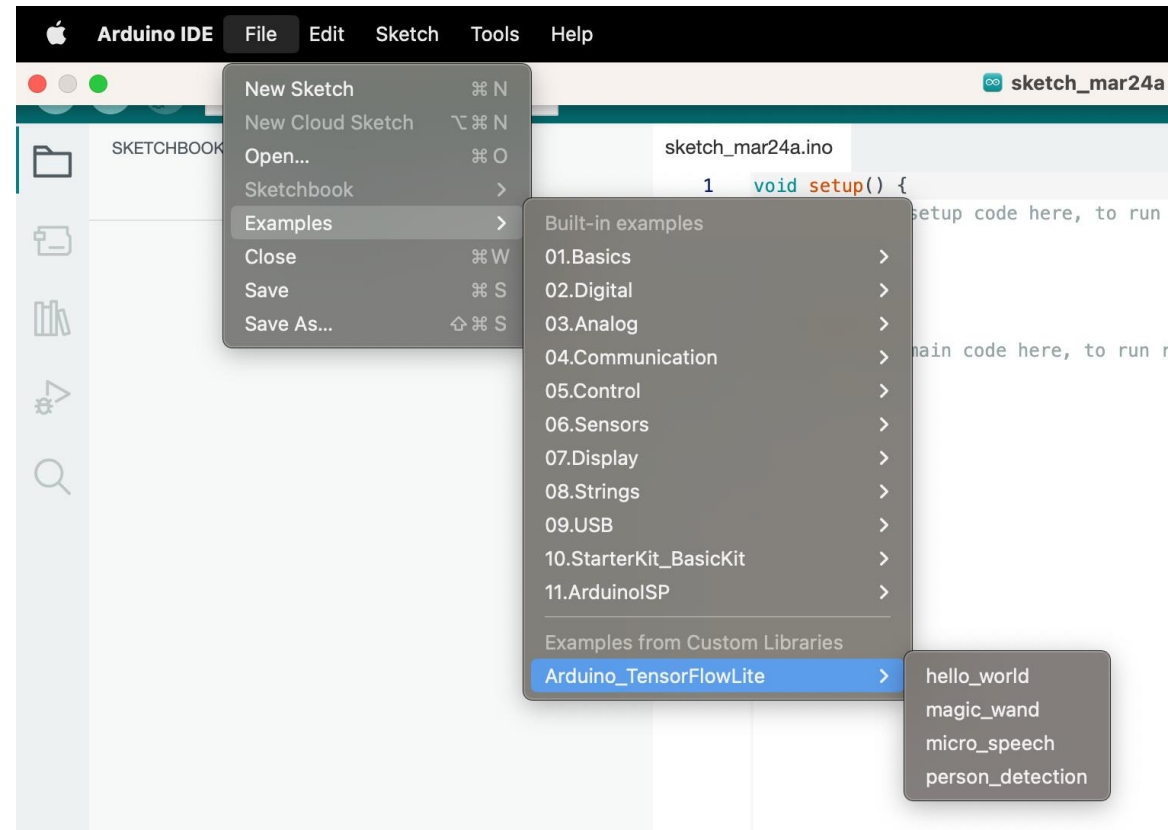


Successfully installed library from tflite-micro-arduino-examples-main.zip archive

Setting up Arduino Desktop IDE

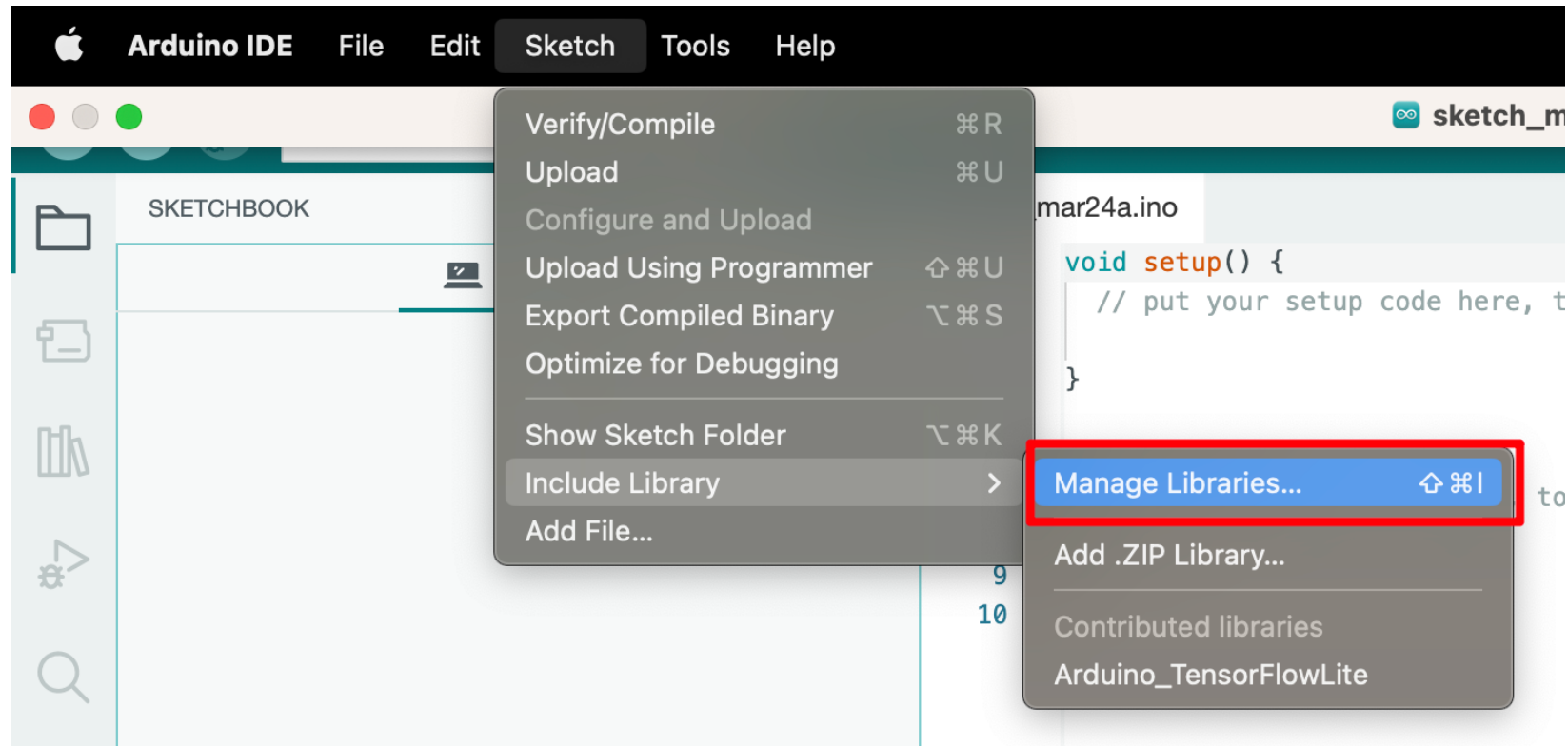
1. TensorFlow Lite Micro Library for Arduino

Now you can see...



Setting up Arduino Desktop IDE

2. Harvard_TinyMLx

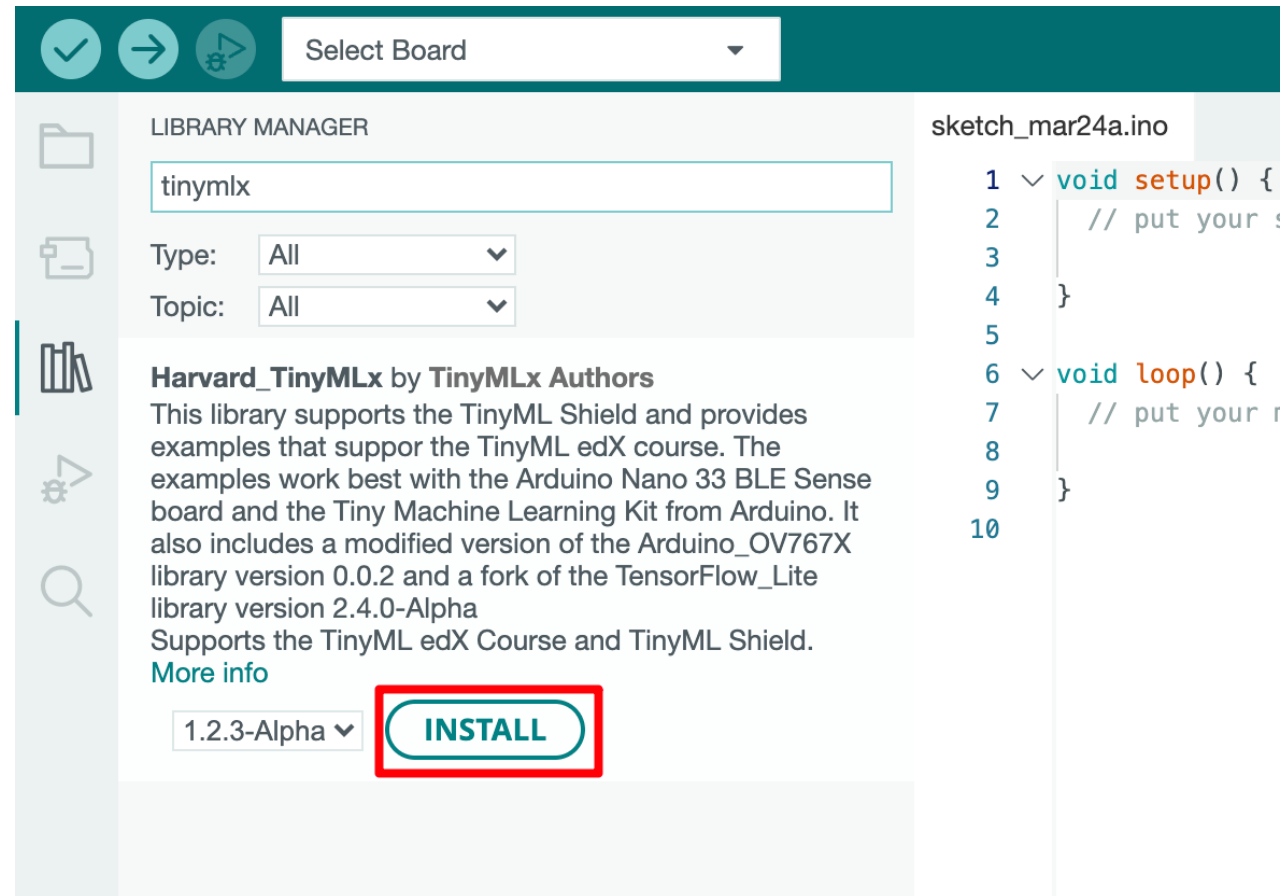


Setting up Arduino Desktop IDE

2. Harvard_TinyMLx

Search for tinymlx

And install!

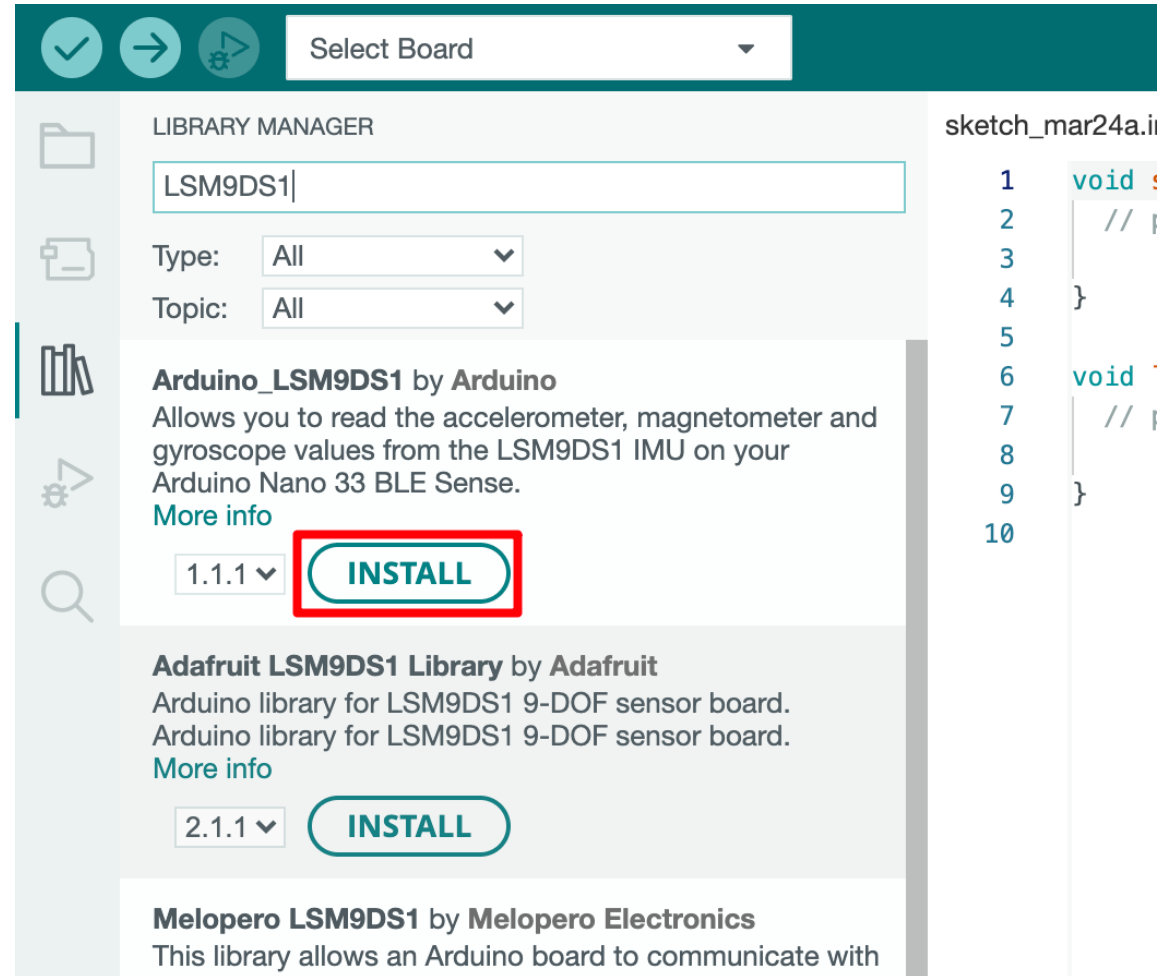


Setting up Arduino Desktop IDE

3. Arduino_LSM9DS1

Search for LSM9DS1

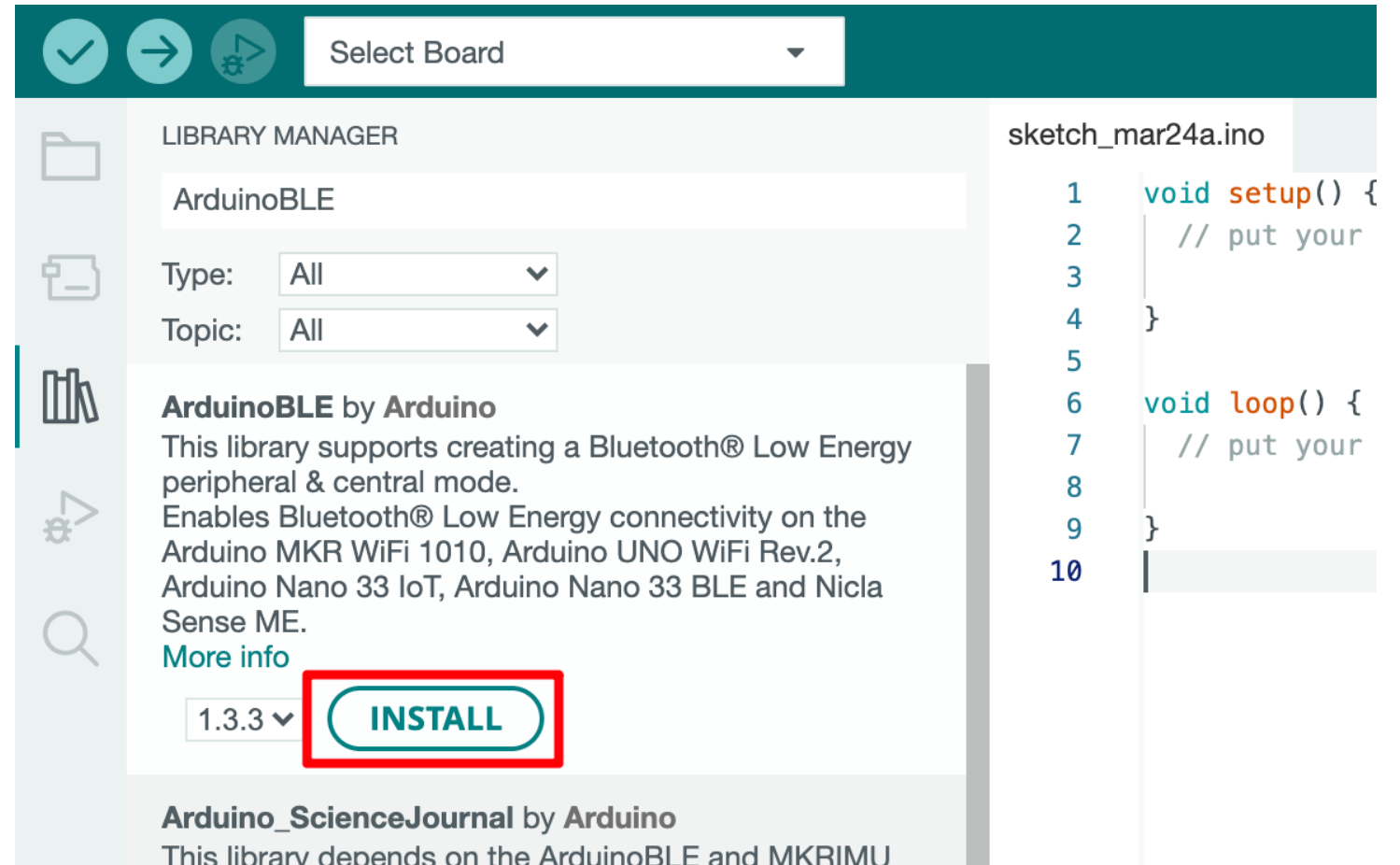
And install!



Setting up Arduino Desktop IDE

4. ArduinoBLE

Search for ArduinoBLE
And install!

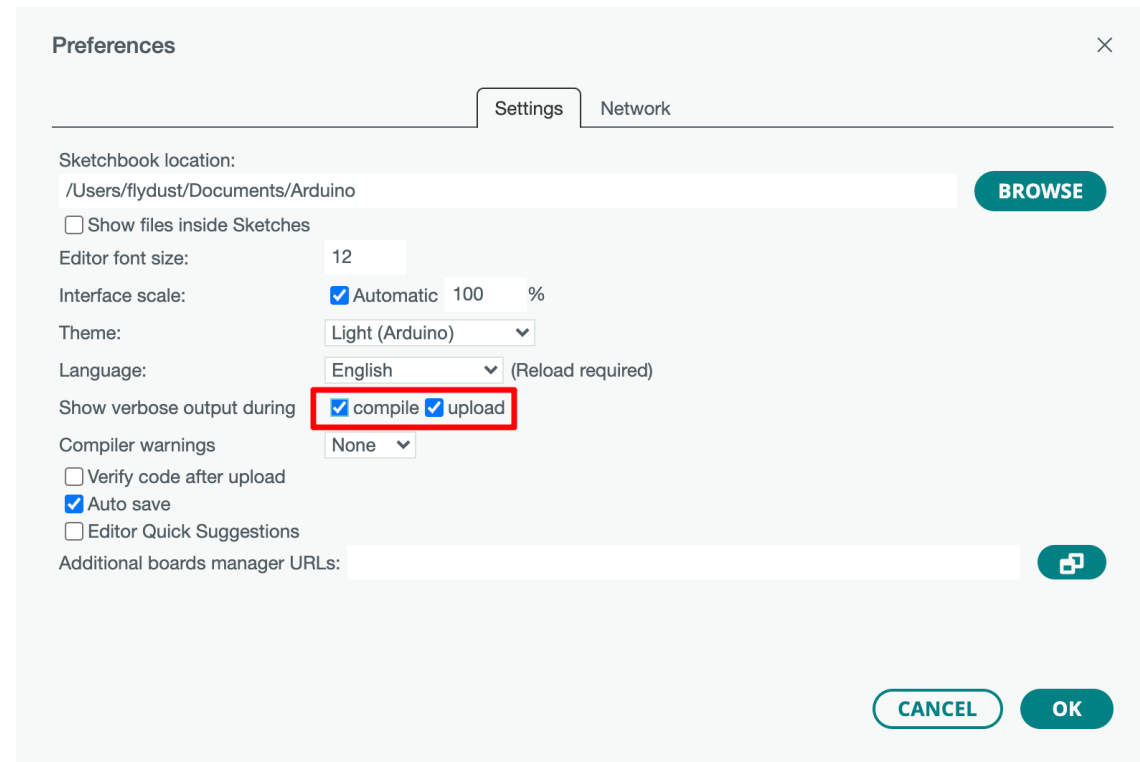
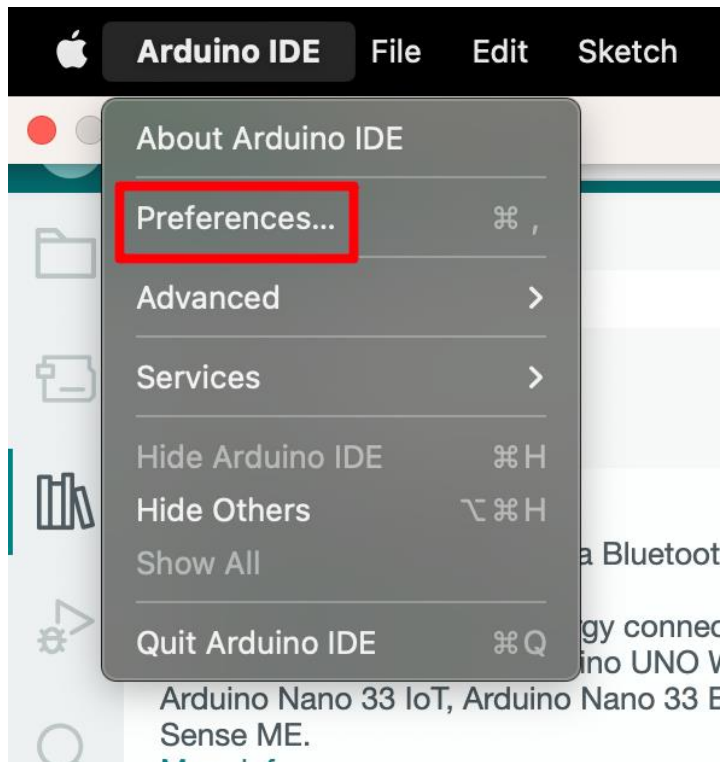


The screenshot shows the Arduino IDE interface. At the top, there's a 'Select Board' dropdown menu. Below it, the 'LIBRARY MANAGER' tab is active. A search bar contains the text 'ArduinoBLE'. The results show the 'ArduinoBLE' library by 'Arduino'. The description states: 'This library supports creating a Bluetooth® Low Energy peripheral & central mode. Enables Bluetooth® Low Energy connectivity on the Arduino MKR WiFi 1010, Arduino UNO WiFi Rev.2, Arduino Nano 33 IoT, Arduino Nano 33 BLE and Nicla Sense ME.' Below the description is a 'More info' link and a version dropdown set to '1.3.3'. A red rectangle highlights the 'INSTALL' button. To the right, the 'sketch_mar24a.ino' file is open, showing the following code:

```
1 void setup() {  
2   // put your  
3  
4 }  
5  
6 void loop() {  
7   // put your  
8  
9 }  
10
```


Setting up Arduino Desktop IDE

- Setting the preferences:
- show verbose output during: Compile and Upload



Task 2 — Arduino IDE and Board Setup

Goal: Install Arduino IDE, install the correct board package, and select the correct board and port for the Arduino Nano 33 BLE Sense Rev2.

What you need

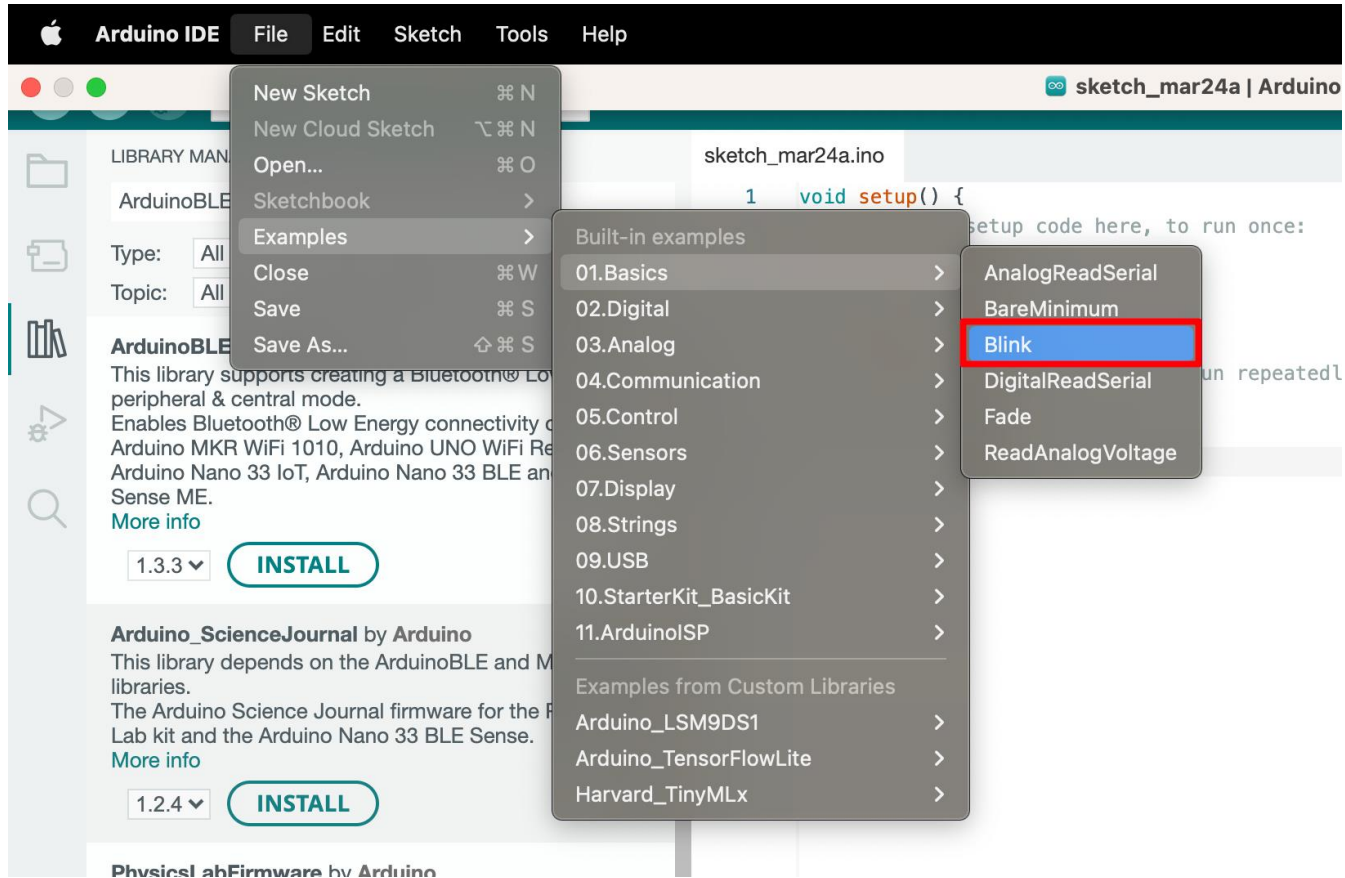
- Arduino Nano 33 BLE Sense Rev2 connected over USB
- Internet connection
- Arduino IDE 2 installed on the computer

Step-by-step instructions

1. Open Arduino IDE 2. (If Arduino IDE is not installed yet, download and install it from the official Arduino page.)
2. With the board connected, wait a few seconds and look for the board selector near the top of the IDE window.
3. If Arduino IDE detects the board and asks to install a missing board package, click **Yes**.
4. If no prompt appears, open **Boards Manager** from the left sidebar.
5. Search for **Mbed OS Nano** and Install the package that contains support for Nano 33 BLE family boards.
6. After installation, select the board as **Arduino Nano 33 BLE Sense** or the corresponding entry shown by the IDE for the connected Rev2 board.
7. Open the board/port selector (under **Tools** tab) and select the USB port associated with the board.
8. If no port appears, unplug and reconnect the board.
9. If the board is still not detected, double-tap the reset button and check whether a new port appears.
10. Once the board and port are selected, leave the board connected for the next section.

The Arduino Blink Example

Open the *Blink.ino* sketch, which you can find via the File drop-down menu. Navigate, as follows: **File** → **Examples** → **01.Basics** → **Blink**.



The Arduino Blink Example

// the setup function runs once when you press reset or power the board

```
void setup() {  
  // initialize digital pin LED_BUILTIN as an output. pinMode(LED_BUILTIN, OUTPUT);
```

```
}  
// the loop function runs over and over again forever
```

```
void loop() {  
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level) delay(1000); // wait for a second  
  digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW delay(1000); // wait for a second  
}
```

The Arduino Blink Example



```
[=====] 85% (18/21 pages)
[=====] 90% (19/21 pages)
[=====] 95% (20/21 pages)
[=====] 100% (21/21 pages)
Done in 3.342 seconds
```

Task 3: Blink Test

Goal: Verify that the IDE can compile and upload a sketch successfully and that the onboard LED responds as expected.

What you need

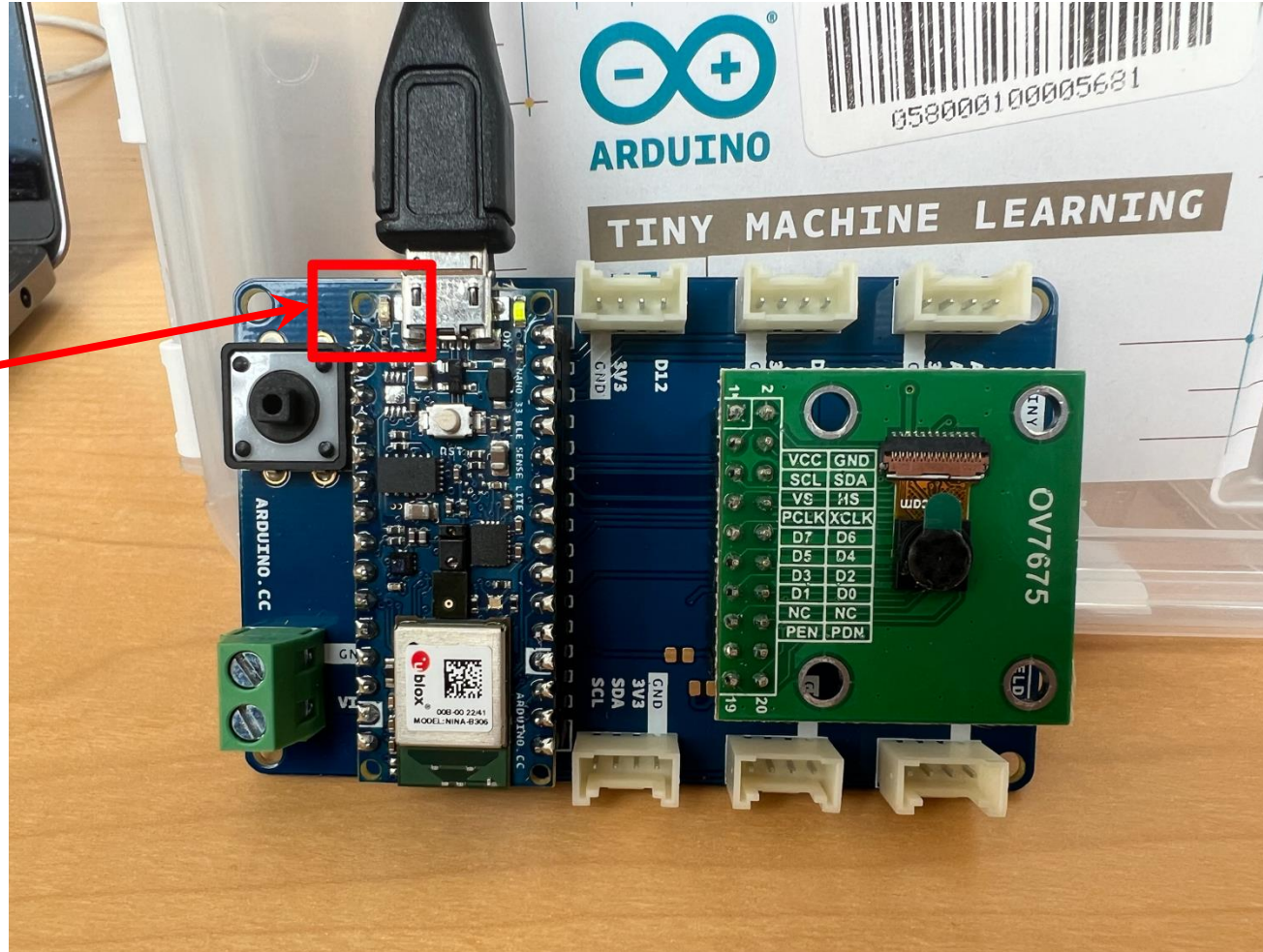
- Board connected over USB
- Correct board and port already selected in Arduino IDE

Step-by-step instructions

1. In Arduino IDE, open **File** → **Examples** → **01.Basics** → **Blink**.
2. Confirm that the correct board and port are still selected.
3. Click **Verify** first and wait for compilation to finish.
4. If compilation succeeds, click **Upload**.
5. Wait for the upload to complete.
6. Observe the onboard LED and confirm that it blinks on and off repeatedly.
7. If upload fails, press the reset button once and try again.
8. If needed, double-tap reset to enter bootloader mode and upload again.
9. Leave the Blink sketch on the board until the behavior is clearly verified.

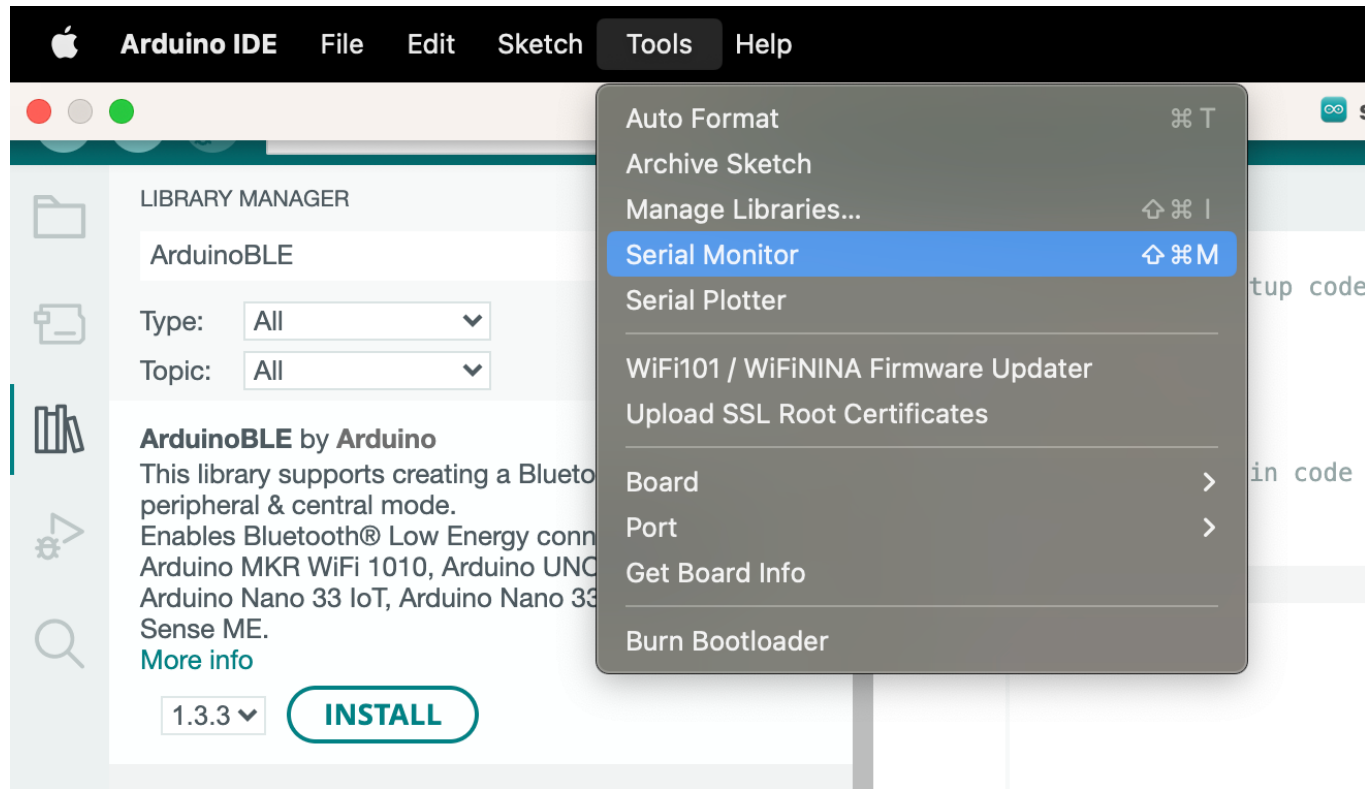
The Arduino Blink Example

This LED should
be blinking

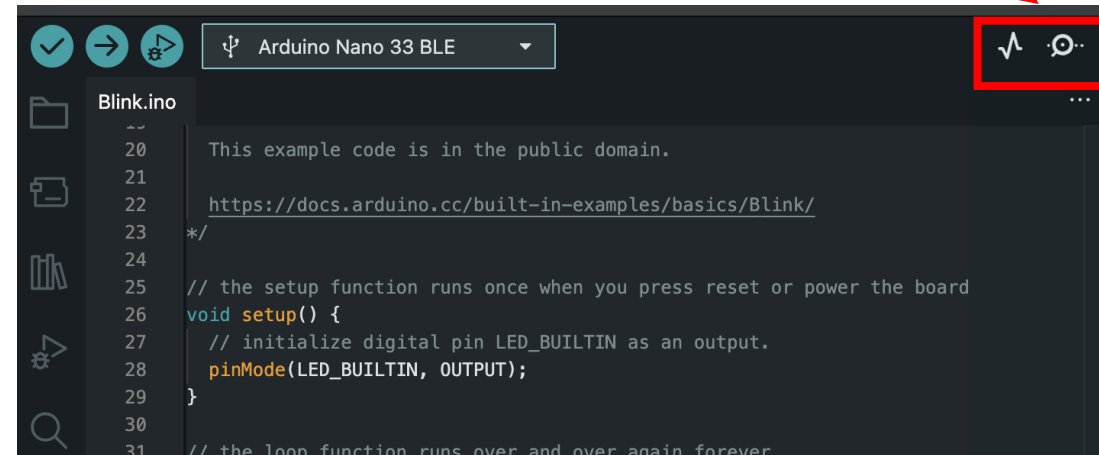


The Serial Monitor and Plotter

You can open up the Serial Monitor (and Serial Plotter) by navigating through the menu to **Tools** → **Serial Monitor** or **Tools** → **Serial Plotter**.



OR click on the following icons in IDE



Task 4 — Serial Monitor / Serial Plotter

Goal: Make sure you can print values from the board; open Serial Monitor correctly and use Serial Plotter for later sensor verification tasks.

What you need

- Board connected over USB
- Arduino IDE already working

Step-by-step instructions

1. Open a new sketch in Arduino IDE.
2. Copy and paste the following code:

```
void setup() {  
    Serial.begin(115200);  
    delay(1500);  
    Serial.println("Serial test started");  
}  
  
void loop() {  
    static int count = 0;  
    Serial.print("Count: ");  
    Serial.println(count);  
    count++;  
    delay(1000);  
}
```

Task 4 — Serial Monitor / Serial Plotter

3. Verify and upload the sketch.
4. Open **Serial Monitor**.
5. Set the baud rate to **115200**.
6. Confirm that the message “**Serial test started appears**”, followed by increasing counter values.
7. Close Serial Monitor.
8. Replace the code with the following simple plotting example:

```
void setup() {  
  Serial.begin(115200);  
  delay(1500);  
}
```

```
void loop() {  
  static int x = 0;  
  int y = (x % 20);  
  Serial.println(y);  
  x++;  
  delay(100);  
}
```

9. Verify and upload the new sketch.
10. Open Serial Plotter.
11. Confirm that a changing plotted trace appears.

What you should submit:

- A screenshot of Serial Monitor showing the counter output
- A screenshot of Serial Plotter showing the live trace

Task 5 — Microphone Verification

Goal: Verify that the onboard digital microphone on the Arduino Nano 33 BLE Sense Rev2 is working and that students can observe changing audio-related values in Serial Plotter.

What you need

- Board connected over USB
- Arduino IDE working
- Ability to open Serial Plotter

Library setup

1. Open **Library Manager**.
2. Search for **PDM**.
3. If it is not already available, install it.

Step-by-step instructions

1. Open a new sketch.
2. Copy and paste the following code:

```
#include <PDM.h>

short sampleBuffer[256];
volatile int samplesRead = 0;

void onPDMdata() {
    int bytesAvailable = PDM.available();
    PDM.read(sampleBuffer, bytesAvailable);
    samplesRead = bytesAvailable / 2;
}

void setup() {
    Serial.begin(115200);
    delay(1500);

    PDM.onReceive(onPDMdata);

    if (!PDM.begin(1, 16000)) {
        Serial.println("Failed to start PDM microphone.");
        while (1);
    }
}

void loop() {
    if (samplesRead) {
        long sum = 0;
        for (int i = 0; i < samplesRead; i++) {
            sum += abs(sampleBuffer[i]);
        }
        int level = sum / samplesRead;
        Serial.println(level);
        samplesRead = 0;
    }
}
```



Task 5 — Microphone Verification

3. Verify and upload the sketch.
4. Open **Serial Plotter** and set the baud rate to **115200**.
5. Stay quiet for a few seconds and observe the plotted values.
6. Speak, clap softly, or tap the desk near the board.
7. Confirm that the plotted values increase or change clearly when sound is present.
8. If needed, also open **Serial Monitor** briefly to confirm the board is printing numeric values.

What you should submit:

- *A screenshot of Serial Plotter while the environment is relatively quiet*
- *A screenshot of Serial Plotter while speaking, clapping, or tapping near the board*
- *One short sentence describing what action caused the clearest change in the plotted signal*

Task 6A — IMU Verification

Goal: Verify that the onboard IMU is working by reading accelerometer values and observing clear changes when the board is tilted or moved.

What students need

- Board connected over USB
- Arduino IDE working
- Ability to open Serial Monitor

Library setup

1. Open **Library Manager**.
2. Search for **Arduino_BMI270_BMM150**.
3. Install the library if needed.

Step-by-step instructions

1. Open a new sketch.
2. Copy and paste the following code:

```
#include <Arduino_BMI270_BMM150.h>

void setup() {
  Serial.begin(115200);
  delay(1500);

  if (!IMU.begin()) {
    Serial.println("Failed to initialize IMU.");
    while (1);
  }

  Serial.println("Accelerometer test started");
  Serial.println("ax,ay,az");
}

void loop() {
  float x, y, z;

  if (IMU.accelerationAvailable()) {
    IMU.readAcceleration(x, y, z);
    Serial.print(x, 3);
    Serial.print(",");
    Serial.print(y, 3);
    Serial.print(",");
    Serial.println(z, 3);
  }

  delay(100);
}
```

Task 6A — IMU Verification

3. Verify and upload the sketch.
4. Open **Serial Monitor** at **115200** baud first.
5. Confirm that comma-separated accelerometer values are printed continuously.
6. Slowly tilt the board in different directions.
7. Confirm that the values change clearly as the board orientation changes.
8. If desired, open **Serial Plotter** instead and observe the three channels changing over time.

What you should submit:

- *A screenshot of Serial Monitor showing changing accelerometer values*
- *A screenshot of Serial Plotter showing the three changing channels, if Plotter is used*
- *One short sentence explaining which movement produced the clearest change*

Task 6B — Gyroscope Verification

Goal

Verify that the onboard gyroscope is working by reading angular velocity values and observing board is rotated.

What students need

- Board connected over USB
- Arduino IDE working
- Ability to open Serial Monitor or S

Library setup

- 1.Open **Library Manager**.
- 2.Search for **Arduino_BMI270_BM**
- 3.Install the library if needed.

Step-by-step instructions

- 1.Open a new sketch.
- 2.Copy and paste the following code:

```
#include <ArduinoIMU.h>
void loop() {
    float x, y, z;

    void setup() {
        Serial.begin(115200);
        delay(1500);

        if (!IMU.begin()) {
            Serial.print("IMU not found\n");
            while (1);
        }

        Serial.println("Gyroscope Test\n");
        Serial.println("-----");
        delay(100);

        if (IMU.gyroscopeAvailable()) {
            IMU.readGyroscope(x, y, z);
            Serial.print(x, 3);
            Serial.print(", ");
            Serial.print(y, 3);
            Serial.print(", ");
            Serial.print(z, 3);
            Serial.println();
        }
    }
}
```


Task 6B — Gyroscope Verification

3. Verify and upload the sketch.
4. Open **Serial Monitor** at **115200** baud.
5. Confirm that three comma-separated gyroscope values are printed continuously.
6. Rotate the board slowly around different axes.
7. Confirm that the values change clearly during rotation and move closer to zero when the board is still.
8. If desired, use **Serial Plotter** to visualize the three channels.

What you should submit:

- *A screenshot of Serial Monitor showing changing gyroscope values*
- *A screenshot of Serial Plotter showing the three channels, if Plotter is used*
- *One short sentence describing which rotation produced the clearest change*

Task 6C — Magnetometer Verification

Goal

Verify that the onboard magnetometer is working by reading magnetic field values and observing changes when the board is rotated or moved near metallic objects.

What students need

- Board connected over USB
- Arduino IDE working
- Ability to open Serial Monitor or Serial Plotter

Library setup

1. Open **Library Manager**.
2. Search for **Arduino_BMI270_BMM**.
3. Install the library if needed.

Step-by-step instructions

1. Open a new sketch.
2. Copy and paste the following code:

```
void loop() {  
    float x, y, z;  
  
    if (IMU.magneticFieldAvailable()) {  
        IMU.readMagneticField(x, y, z);  
        Serial.print(x, 3);  
        Serial.print(",");  
        Serial.print(y, 3);  
        Serial.print(",");  
        Serial.println(z, 3);  
    }  
  
    delay(200);  
}
```

Task 6C — Magnetometer Verification

3. Verify and upload the sketch.
4. Open **Serial Monitor** at **115200** baud.
5. Confirm that three comma-separated magnetic field values are printed continuously.
6. Rotate the board slowly in different directions.
7. Optionally bring the board near a metallic object or electronic device and observe whether the values change.
8. If desired, use **Serial Plotter** to visualize the three channels.

What you should submit:

- *A screenshot of Serial Monitor showing changing magnetometer values*
- *A screenshot of Serial Plotter showing the three channels, if Plotter is used*
- *One short sentence describing what action produced the clearest change*

Task 7 — Humidity and Temperature Verification

Goal

Verify that the onboard humidity and temperature sensor is working and that students can observe stable readings and intentional changes in the environment.

What students need

- Board connected over USB
- Arduino IDE working
- Ability to open Serial Monitor

Library setup

1. Open **Library Manager**
2. Search for **Arduino Humidity and Temperature**
3. Install the library if not installed

Step-by-step instructions

1. Open a new sketch.
2. Copy and paste the following code:

```
#include <Arduino_HS300x.h>

void setup() {
  Serial.begin(115200);
  delay(1500);

  Serial.print("Humidity and Temperature sensor started");
}

void loop() {
  float temperature = HS300x.readTemperature();
  float humidity = HS300x.readHumidity();

  Serial.print("Temperature (C): ");
  Serial.print(temperature, 2);
  Serial.print(" | Humidity (%): ");
  Serial.println(humidity, 2);

  delay(1000);
}
```


Task 7 — Humidity and Temperature Verification

3. Verify and upload the sketch.
4. Open **Serial Monitor** at **115200** baud.
5. Observe the temperature and humidity readings for a few seconds.
6. Gently place your hand near the board or breathe lightly toward the sensor from a short distance.
7. Check whether the humidity and/or temperature readings change.
8. Let the board sit still again and observe whether the values settle.

What you should submit:

- *A screenshot of Serial Monitor showing stable humidity and temperature values*
- *A second screenshot showing changed values after introducing a nearby hand or gentle breath*
- *One short sentence describing what action produced the clearest change*

Task 8 — Barometric Pressure Verification

Goal

Verify that the onboard barometric pressure sensor is working and that students can read stable pressure values from the board.

What students need

- Board connected over USB
- Arduino IDE working
- Ability to open Serial Monitor

Library setup

1. Open **Library Manager**
2. Search for **Arduino_LPS22HB**
3. Install the library if not installed

```
#include <Arduino_LPS22HB.h>

void loop() {
    float pressure = BAR0.readPressure();
    float temperature = BAR0.readTemperature();

    Serial.print("Pressure (kPa): ");
    Serial.print(pressure, 3);
    Serial.print(" | Temperature (C): ");
    Serial.println(temperature, 2);

    delay(1000);
}
```

Step-by-step instructions

1. Open a new sketch.
2. Copy and paste the following code:

Task 8 — Barometric Pressure Verification

3. Verify and upload the sketch.
4. Open **Serial Monitor** at **115200** baud.
5. Observe the pressure and temperature readings for several seconds.
6. Confirm that the pressure value is stable and updates continuously.
7. Optionally raise the board slightly or move it to a different location in the room and observe whether the value changes slightly.
8. Use this section mainly to confirm correct sensor operation rather than expecting a large classroom-scale pressure change.

What you should submit:

- *A screenshot of Serial Monitor showing pressure and temperature values*
- *One short sentence stating whether the pressure reading was mostly stable or changed slightly during testing*



Task 9A — Proximity Verification

Goal

Verify that the onboard proximity sensor is working and students can observe a clear change in the reported value as the hand moves closer to the board.

What students need

- Board connected over USB
- Arduino IDE working
- Ability to open Serial Monitor

Library setup

1. Open **Library Manager**.
2. Search for **Arduino_APDS9960**.
3. Install the library if needed.

Step-by-step instructions

1. Open a new sketch.
2. Copy and paste the following code:

```
#include <Arduino_APDS9960.h>

void setup() {
  Serial.begin(115200);
  delay(1500);

  if (!APDS.begin()) {
    Serial.println("Failed to initialize APDS9960 sensor.");
    while (1);
  }

  Serial.println("Proximity test started");
}

void loop() {
  if (APDS.proximityAvailable()) {
    int proximity = APDS.readProximity();
    Serial.print("Proximity: ");
    Serial.println(proximity);
  }

  delay(100);
}
```

Task 9A — Proximity Verification

3. Verify and upload the sketch.
4. Open **Serial Monitor** at **115200** baud.
5. Hold your hand far from the board and observe the reported values.
6. Slowly move your hand closer to the sensor area on the board.
7. Confirm that the reported proximity value changes clearly.
8. Move your hand away again and observe whether the reading returns toward the earlier value.

What you should submit:

- *A screenshot of Serial Monitor with the hand far from the board*
- *A screenshot of Serial Monitor with the hand close to the board*
- *One short sentence describing how the value changed*

Task 9B — Gesture Verification

Goal

Optionally verify that the APDS9960 sensor can detect simple hand gestures such as left, right, up and down.

What students need

- Board connected over USB
- Arduino IDE working
- Ability to open Serial Monitor

Library setup

1. Open **Library Manager**.
2. Search for **Arduino_APDS9960**.
3. Install the library if needed.

```
void loop() {  
  # if (APDS.gestureAvailable()) {  
    int gesture = APDS.readGesture();  
  
    if (gesture == GESTURE_UP) {  
      Serial.println("UP");  
    } else if (gesture == GESTURE_DOWN) {  
      Serial.println("DOWN");  
    } else if (gesture == GESTURE_LEFT) {  
      Serial.println("LEFT");  
    } else if (gesture == GESTURE_RIGHT) {  
      Serial.println("RIGHT");  
    }  
  }  
}
```

Step-by-step instructions

1. Open a new sketch.
2. Copy and paste the following code:

Task 9B — Gesture Verification

3. Verify and upload the sketch.
4. Open **Serial Monitor** at **115200** baud.
5. Move your hand slowly near the sensor area using simple left, right, up, and down motions.
6. Check whether the board prints gesture labels such as “LEFT”, “RIGHT”, “UP”, or “DOWN”.
7. Try a few times if needed, because gesture sensing may depend on hand distance, angle, speed, and ambient conditions.

What you should submit:

- *One screenshot showing a recognized gesture in Serial Monitor*
- *One short sentence stating which gesture was recognized most reliably*

Task 9C — Ambient Light and RGB Color Verification

Goal

Verify that the APDS9960 sensor can read color-related channel values and has ambient light capability.

What students need

- Board connected over USB
- Arduino IDE working
- Ability to open Serial Monitor

Library setup

1. Open **Library Manager**.
2. Search for **Arduino_APDS9960**.
3. Install the library if needed.

Step-by-step instructions

1. Open a new sketch.
2. Copy and paste the following code:

```
#include <Arduino_APDS9960.h>

// ...

void loop() {
  int r, g, b, c;

  if (APDS.colorAvailable()) {
    APDS.readColor(r, g, b, c);
    Serial.print(r);
    Serial.print(",");
    Serial.print(g);
    Serial.print(",");
    Serial.print(b);
    Serial.print(",");
    Serial.println(c);
  }

  delay(200);
}
```

```
  initialize APDS9960 sensor.");

  // ...

  color test started");
```

Task 9C — Ambient Light and RGB Color Verification

3. Verify and upload the sketch.
4. Open **Serial Monitor** at **115200** baud.
5. Observe the baseline “r”, “g”, “b”, “clear” values under normal room lighting.
6. Change the lighting conditions by shading the sensor slightly with your hand and then exposing it to brighter light again.
7. If available, bring a colored object such as a red, green, or blue item near the sensor and observe whether one or more RGB channels respond differently.
8. Confirm that the clear channel and RGB values change in a reasonable way when lighting conditions or nearby colors change.

What you should submit:

- *A screenshot of Serial Monitor under baseline room lighting*
- *A screenshot of Serial Monitor after changing lighting conditions or presenting a colored object*
- *One short sentence describing which change produced the clearest difference*

Task 10 — Smart Workspace Situation Classifier

Context: You are building a simple rule-based “workspace awareness” system for a desk device. The device must infer the current desk situation using multiple onboard sensing modalities and print a clear situation label to the Serial Monitor.

Objective: Design and implement an Arduino program that classifies the desk into **one of four predefined situations** using the following **four sensing modalities**:

- **Audio activity** from the microphone
- **Ambient brightness** from the APDS9960 light sensor
- **Physical motion** from the IMU
- **User presence near the board** from the APDS9960 proximity sensor

Required situation labels:

Your program must classify exactly these four situations:

1. QUIET_BRIGHT_STEADY_FAR
2. NOISY_BRIGHT_STEADY_FAR
3. QUIET_DARK_STEADY_NEAR
4. NOISY_BRIGHT_MOVING_NEAR.

Task 10 — Serial Monitor / Serial Plotter

What your program must do

Your code must:

- read all four sensing modalities continuously
- compute one binary decision for each modality
- combine those decisions using **rule-based logic only**
- print the final situation label to Serial Monitor

*TinyML models are **not allowed** for this question. The goal is sensor fusion through simple logic.*

Serial Monitor requirements

For each update cycle, print **three lines** in the following format:

```
raw,mic=<value>,clear=<value>,motion=<value>,prox=<value>  
flags,sound=<0/1>,dark=<0/1>,moving=<0/1>,near=<0/1>  
state,<FINAL_LABEL>
```

Meaning of the printed quantities

- `mic` = microphone activity level
- `clear` = ambient light / clear channel value
- `motion` = chosen motion metric from the IMU
- `prox` = proximity reading
- `sound`, `dark`, `moving`, `near` = thresholded binary decisions
- `FINAL_LABEL` = one of the four required labels

Task 10 — Serial Monitor / Serial Plotter

Testing requirement:

Demonstrate the program in **all four required situations** and capture Serial Monitor evidence for each one.

What you should submit:

- *Arduino code in GitHub*
- *GitHub link in your lab submission*
- *four Serial Monitor screenshots, one for each required situation*
- *a short explanation of how you selected your thresholds*

Task 11 — Serial Monitor / Serial Plotter

Context: You are building a simple rule-based monitoring system for a compact smart device placed in a real indoor environment. The device must detect short environmental or nearby-object changes and report them as event labels.

Objective: Design and implement an Arduino program that detects **one of four predefined events** using the following sensing modalities:

- **Humidity** from the HS3003 sensor
- **Temperature** from the HS3003 sensor
- **Magnetic disturbance** from the magnetometer
- **Ambient/color response** from the APDS9960 RGB/clear channels

Required event labels

Your program must detect exactly these four events:

- 1.BASELINE_NORMAL
- 2.BREATH_OR_WARM_AIR_EVENT
- 3.MAGNETIC_DISTURBANCE_EVENT
- 4.LIGHT_OR_COLOR_CHANGE_EVENT

Task 11 — Serial Monitor / Serial Plotter

What your program must do

Your code must:

- read all required sensing modalities continuously
- detect event conditions using **rule-based logic only**
- print a single final event label to Serial Monitor
- avoid repeated rapid re-triggering of the same event by using a short cooldown or debounce logic

TinyML models are not allowed for this question. The goal is sensor fusion through simple logic.

Serial Monitor requirements

For each update cycle, print **three lines** in the following format:

Meaning of the printed quantities

```
raw,rh=<value>,temp=<value>,mag=<value>,r=<value>,g=<value>,b=<value>,clear=<value>  
flags,humid_jump=<0/1>,temp_rise=<0/1>,mag_shift=<0/1>,light_or_color_change=<0/1>  
event,<FINAL_LABEL>
```

- rh = relative humidity
- temp = temperature
- mag = chosen magnetic-field metric
- r, g, b, clear = APDS9960 color/light channels
- humid_jump, temp_rise, mag_shift, light_or_color_change = thresholded event indicators
- FINAL_LABEL = one of the four required event labels

Task 11 — Serial Monitor / Serial Plotter

Testing requirement

Demonstrate the program in **all four required events** and capture Serial Monitor evidence for each one.

What you should submit:

- *Arduino code in GitHub*
- *GitHub link in your lab submission*
- *four Serial Monitor screenshots, one for each required event*
- *a short explanation of how you selected your thresholds and cooldown logic*



[Optional] Testing the OV7675 Camera

1. Open the test_camera.ino sketch, which you can find via the File drop-down menu. Navigate, as follows: **File → Examples → Harvard_TinyMLx → test_camera**.
2. As always, use the Tools drop down menu to select appropriate Port and Board.
3. Then use the rightward arrow next to the 'compile' checkmark to upload / flash the code. If you get the error "Arduino_OV767X.h: No such file or directory" then please go back to Setting up the Software and make sure you install the accelerometer, magnetometry, and gyroscope library! To help you debug please check out our FAQ appendix with answers to the most common errors! Note: if you get an error message that contains something like **'OV7675' was not declared in this scope** this means you already had a conflicting copy of the Arduino_OV767X library installed on your system. We have bundled a forked copy of it with our library so please uninstall the conflicting version.
4. If you get a message that "Your board was not found", this could be due that the MCU was "busy" running the previous code. Click the Reset button twice (you will see the Build in LED blinking slowly). Upload the sketch again.

[Optional] Testing the OV 7675 Camera

5. Open the Serial Monitor you should see:

```
Welcome to the OV7675 test
```

```
Available commands:
```

```
single - take a single image and print out the hexadecimal for each pixel (default)
```

```
live - the raw bytes of images will be streamed live over the serial port
```

```
capture - when in single mode, initiates an image capture
```

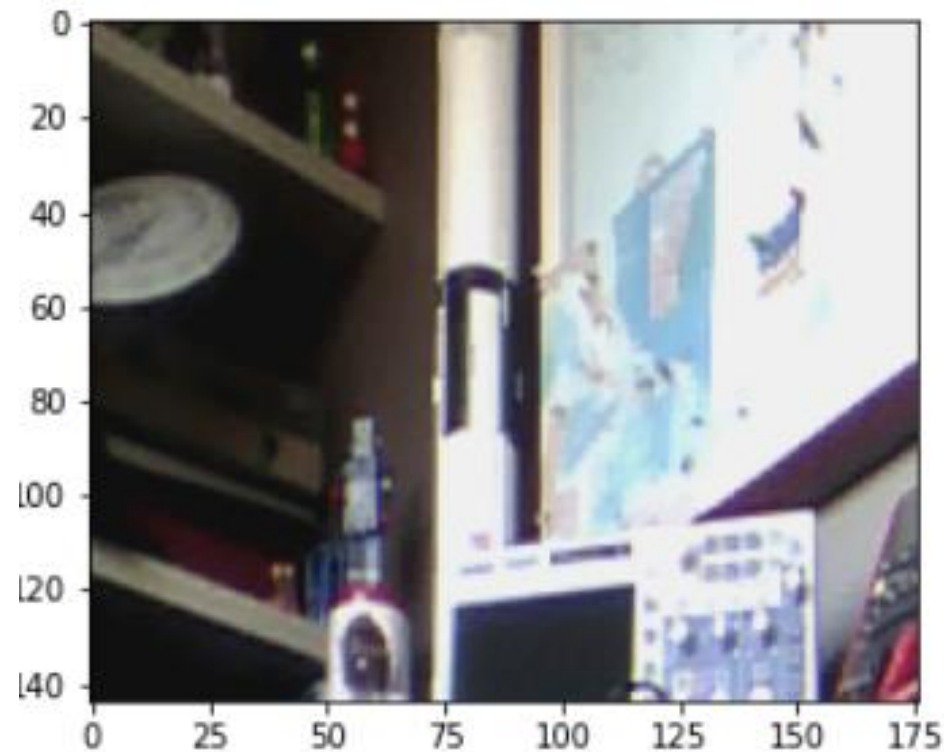
6. Type “single” in the text entry field and press send or hit the enter / return key. The camera is now primed to take a picture. Now you can either text “capture” or press the on-shield button to take a selfie (or photo). To get the photo oriented correctly make sure the “OV7675” text on the camera module (or the “Tiny Machine Learning Shield” on the shield) is oriented such that it would be readable by the subject of the photo (e.g., you if you are taking a selfie). **The camera does not have a large field of view so if you are taking a selfie make sure to hold the camera far away from your face.**



[Optional] Testing the OV 7675 Camera

7. To view your image you will need to copy the sequence of hexadecimal digits that will print to the Serial Monitor and paste them into the notebook:

[*OV7675_Image_Viewer.ipynb*](#).



Lab 2 Final Submission Guidelines

Submission format: Submit **one professionally formatted PDF report** via Canvas.

Your report must include

1. Student information

- Full name
- Student ID

2. Task results

- Include **screenshots of results for Tasks 4 to 11**
- **Present results under the corresponding task headings**
- Ensure all screenshots are clear, readable, and properly labeled

3. Answers to lab questions

1. Provide **complete answers to the questions in Tasks 4 to 11**
2. **Place each answer under its corresponding task**
3. Write clearly and concisely in a professional format

4. GitHub submission

1. Include **one GitHub link** to the repository hosting the **Arduino sketches developed for Tasks 10 and 11**
2. Organize the repository neatly and use meaningful file names

Lab 2 Final Submission Guidelines

Report expectations:

- Use a clean and professional structure
- Clearly separate each task using section headings such as:
 - **Task 4**
 - **Task 5**
 - ...
 - **Task 11**
- For each relevant task, include:
 - result screenshots
 - short discussion / interpretation
 - answers to the associated questions

Important Note:

- Your submission should be a **single PDF report**
- Make sure the **GitHub link is active and accessible**
- Poorly organized reports, missing screenshots, missing answers, or missing GitHub links may lose marks

Lab 2 References

- 1) <https://docs.arduino.cc/resources/datasheets/ABX00069-datasheet.pdf>
- 2) https://tinymt.seas.harvard.edu/assets/other/4D/22.03.11_Marcelo_Rovai_Handout.pdf
- 3) <https://www.arduino.cc/en/software>